

Visualização de conteúdos de

MATEMÁTICA

via biblioteca

MANIM

do Python

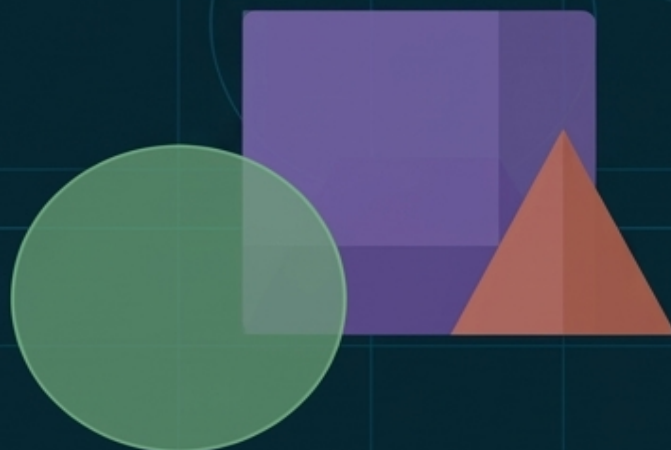
EMILLY MAYRE SOUZA MOURA DA SILVA

Orientador: Prof. Dr. Junior Cesar Alves Soares

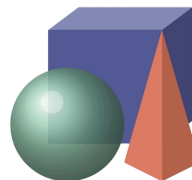
Σ

∞

```
from manim import *  
class MatematicaEmMovimento(Scene):  
    def construct(self):  
        # Visualização em ação...
```



Prefácio



A MATEMÁTICA 9º ano

escolar costuma ser apresentada por meio de símbolos, procedimentos e resultados. Por trás dessas expressões formais existe, no entanto, um conjunto de relações que se tornam mais acessíveis quando podem ser observadas e comparadas em movimento. Uma sequência não é apenas uma lista de números: é um processo gerativo. Uma equação não é apenas uma sentença com incógnita: é uma afirmação de equilíbrio entre duas grandezas. Um ângulo não é apenas um arco entre duas semirretas: é o registro de uma rotação. Representar esses conceitos de forma estática, como o quadro e o livro necessariamente fazem, exige do aluno um esforço de reconstrução mental que muitos não conseguem realizar sozinhos. Este e-book propõe um caminho para reduzir esse esforço: tornar os conceitos visíveis em sua própria natureza dinâmica.

As animações produzidas com a biblioteca Manim que compõem este material não são ilustrações de definições já apresentadas. São representações que tornam perceptíveis aspectos dos objetos matemáticos que permanecem implícitos nas representações estáticas. Quando o aluno vê o ponto se deslocar na reta numérica e cruzar o zero, ele não precisa mais imaginar o que significa somar um número negativo – ele observa. Quando as duas retas surgem no plano e se encontram num único ponto, a solução de um sistema deixa de ser um resultado de cálculo e passa a ser uma interseção visível. Essa diferença entre ver o processo e ver o produto é o fundamento pedagógico de cada animação apresentada aqui.

O material cobre onze descritores do currículo dos anos finais do Ensino Fundamental, selecionados a partir de um critério específico: todos envolvem objetos matemáticos cuja natureza é dinâmica, transformacional ou processual – objetos que se entendem melhor em movimento do que em repouso. Para cada animação,

o professor encontrará quatro orientações articuladas: o que verificar com a turma antes de exhibir, uma atividade de aquecimento para os primeiros minutos da aula, pontos de pausa com perguntas para conduzir durante a exibição, e uma atividade de consolidação para depois. Essa estrutura parte de uma convicção simples: a animação cria as condições para a compreensão, mas não a garante. O que faz a diferença é a mediação do professor antes, durante e depois.

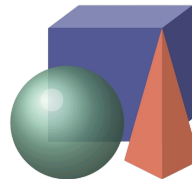
Este material constitui o produto educacional de uma dissertação desenvolvida no âmbito do Mestrado Profissional em Matemática em Rede Nacional (PROFMAT). Ele reflete a proposta do programa de aproximar a pesquisa acadêmica da prática docente – produzindo não apenas conhecimento sobre o ensino de matemática, mas recursos diretamente utilizáveis na sala de aula. Os fundamentos teóricos que orientaram as decisões de design de cada animação estão apresentados na dissertação que acompanha este produto; aqui, eles aparecem na forma de orientações práticas.

Professores podem utilizar este e-book como apoio didático, escolhendo as animações que correspondem aos conteúdos em desenvolvimento com cada turma. O código Python completo de cada animação está disponível em repositório externo indicado ao final, para os professores que queiram adaptar as animações a outros contextos ou produzir novas a partir dos exemplos apresentados.

Se este material contribuir para que um aluno que não conseguia ver o que estava por trás dos símbolos passe a enxergar a relação, o padrão ou o equilíbrio que eles expressam, então terá cumprido o seu propósito.

Boa leitura.

Conteúdo



1	Introdução	1
1.1	O que as animações acrescentam?	2
1.2	Por que o Manim?	2
1.3	Os descritores trabalhados neste e-book	3
1.4	O que você encontrará em cada capítulo de conteúdo	4
1.5	O que este e-book não é	5
1.6	Uma nota sobre o que a pesquisa diz	5
2	Preparando o ambiente	7
2.1	Objetivos do capítulo	7
2.2	Pré-requisitos	7
2.3	Instalação passo a passo	8
2.4	Testando a instalação	10
2.5	Erros comuns e como resolvê-los	11
2.6	Organizando o fluxo de trabalho	11
2.7	Desafio para o professor	11
3	Primeiros passos com objetos para as animações	13
3.1	Objetivos do capítulo	13
3.2	O plano cartesiano do Manim	13
3.3	Objetos básicos (Mobjects)	14
3.3.1	Formas geométricas	15
3.3.2	Texto e expressões matemáticas	15
3.4	Animações fundamentais	17
3.4.1	Criação e desaparecimento	17

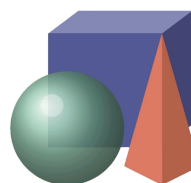
3.4.2	Transformação	18
3.4.3	Movimento	19
3.5	Posicionamento avançado	20
3.5.1	Usando <code>next_to</code>	20
3.5.2	Trabalhando com grupos: <code>VGroup</code>	21
3.6	Exemplo integrador	23
3.7	Resumo dos comandos apresentados	23
3.8	Desafios para o professor	25
4	Design de animações	27
4.1	Planejamento: o storyboard	27
4.2	Controle de ritmo e tempo	28
4.3	Design visual: cor como sinalização	30
4.4	Tipografia nas animações	30
4.5	Exemplo completo: animação do Teorema de Pitágoras	31
4.6	Desafios para o professor	33
5	Criando animações com auxílio de ferramentas de IA	35
5.1	O que a ferramenta faz – e o que ela não faz	35
5.2	Como descrever o que você quer	36
5.3	Do pedido à animação: o caminho completo	37
5.4	Quando o código não funciona	40
5.5	Onde rodar o código	40
5.6	Cada ferramenta tem seu jeito	42
5.7	Comunidade, referências e o que fazer com as animações	43
5.8	Considerações finais	43
6	Localização e movimentação em mapas: D1	45
6.1	Por que animar este conteúdo?	45
6.2	O que a animação mostra	46
6.3	Antes de exibir	46
6.4	O código comentado	47
6.5	Durante a exibição	49
6.6	Depois de ver	49
6.7	Desafios para o professor	50
7	Planificações de sólidos geométricos: D2	53
7.1	Por que animar este conteúdo?	53
7.2	Uma nota sobre cenas 3D no Manim	54

7.3	O que a animação mostra	54
7.4	Antes de exibir	54
7.5	O código comentado	55
7.6	Durante a exibição	58
7.7	Depois de ver	58
7.8	Desafios para o professor	59
8	Ampliação e Redução: D5	61
8.1	Por que animar este conteúdo?	61
8.2	Atenção: proporcionalidade direta e inversa em sessões separadas	62
8.3	O que a animação mostra	62
8.4	Antes de exibir	63
8.5	O código comentado	63
8.6	Durante a exibição	65
8.7	Depois de ver	65
8.8	Desafios para o professor	65
9	Ângulo como giro: D6	67
9.1	Por que animar este conteúdo?	67
9.2	Recurso central: <code>ValueTracker</code> e <code>always_redraw</code>	68
9.3	O que a animação mostra	68
9.4	Antes de exibir	68
9.5	O código comentado	71
9.6	Durante a exibição	72
9.7	Depois de ver	72
9.8	Desafios para o professor	74
10	Coordenadas no plano cartesiano: D9	75
10.1	Por que animar este conteúdo?	75
10.2	O que a animação mostra	76
10.3	Antes de exibir	76
10.4	O código comentado	78
10.5	Durante a exibição	79
10.6	Depois de ver	79
10.7	Desafios para o professor	82
11	Números inteiros na reta numérica: D16	83
11.1	Por que animar este conteúdo?	83
11.2	Em uma sessão ou em duas?	84

11.3	O que a animação mostra	84
11.4	Antes de exibir	85
11.5	O código comentado	87
11.6	Durante a exibição	88
11.7	Depois de ver	88
11.8	Desafios para o professor	91
12	Porcentagem como relação parte-todo: D28	93
12.1	Por que animar este conteúdo?	93
12.2	O que a animação mostra	94
12.3	Antes de exibir	94
12.4	O código comentado	95
12.5	Variação: porcentagem como setor circular	97
12.6	Durante a exibição	99
12.7	Depois de ver	99
12.8	Desafios para o professor	99
13	Proporcionalidade: D29	101
13.1	Por que animar este conteúdo?	101
13.2	Atenção: proporcionalidade direta e inversa em sessões separadas . .	102
13.3	O que a animação mostra	102
13.4	Antes de exibir	103
13.5	O código comentado	103
13.6	Proporcionalidade inversa: código separado	105
13.7	Durante a exibição	107
13.8	Depois de ver	107
13.9	Desafios para o professor	107
14	Descobrimos padrões: D32	109
14.1	Por que animar este conteúdo?	109
14.2	Antes de exibir	110
14.3	Sequências numéricas	110
14.4	Sequências figurativas	114
14.5	Durante a exibição	117
14.6	Depois de ver	117
14.7	Desafios para o professor	117

15 Equações e inequações do 1º grau: D33	119
15.1 Antes de exhibir	119
15.2 Equações na balança	120
15.3 Inequações na reta numérica	123
15.4 Desafios para o professor	126
16 Sistemas de equações e representação geométrica: D34 e D35	127
16.1 Antes de exhibir	127
16.2 Do problema para as retas	128
16.3 Os três casos: uma solução, infinitas, nenhuma	131
16.4 A ponte entre D34 e D35	135
16.5 Desafios para o professor	135
17 Outros tópicos do currículo	137
17.1 Funções afim e quadrática	138
17.2 Teorema de Pitágoras	141
17.3 Juros compostos	143
17.4 Probabilidade	145
Sobre os Autores	150

Introdução



POR QUE ESTE *e-book* existe? Se você está com este material em mãos, é muito provável que já tenha vivido uma cena como esta: você explica um conceito no quadro, os alunos acompanham com os olhos, resolvem alguns exercícios mecânicos, mas, na hora de aplicar a mesma ideia em um problema diferente, surge aquele olhar vago. A conta até sai, mas a compreensão do que está por trás dela – a relação, o padrão, o significado – parece escorregar entre os dedos.

Não se trata de falta de esforço, seu ou deles. O desafio é estrutural: a Matemática lida com objetos que não existem no mundo físico. Ninguém nunca encontrou uma equação andando pela rua. Números, variáveis, relações algébricas são abstrações que precisam ser construídas mentalmente. E construí-las exige representações.

O quadro-negro, o giz, o livro didático – ferramentas consagradas e indispensáveis – oferecem representações estáticas. Mas o pensamento matemático, especialmente quando estamos falando de regularidades, de crescimento de padrões, de equivalência entre expressões, é essencialmente dinâmico. Uma sequência numérica não é apenas uma lista de números; é um processo, uma regra que gera termos. Uma equação não é apenas uma sentença com incógnita; é uma afirmação de equilíbrio entre duas grandezas. Um sistema de equações não é apenas um conjunto de linhas; é um ponto de encontro, uma interseção.

Capturar esse dinamismo com recursos estáticos é como tentar explicar o voo de um pássaro com uma fotografia. A foto mostra o pássaro, mas não o voo. É aí que as animações entram.

1.1 O que as animações acrescentam?

Animar um conceito matemático não é “enfeitar” a aula ou modernizar pelo modernizar. É, antes de tudo, torná-lo visível em sua própria natureza. Quando você mostra uma sequência de figuras se formando passo a passo, o aluno não vê apenas o resultado – vê o processo de construção da regularidade. Quando você traça uma reta no plano cartesiano e, em seguida, traça outra que a cruza, o aluno não apenas localiza o ponto de interseção – ele vê, literalmente, o momento em que as duas condições do problema são satisfeitas ao mesmo tempo.

Há uma diferença fundamental entre dizer “a solução é onde as retas se encontram” e mostrar, com movimento, as duas retas sendo traçadas até que seus caminhos se cruzem. No primeiro caso, a informação é recebida. No segundo, ela é experimentada.

Animações não substituem a explicação do professor, nem o esforço do aluno em raciocinar. Mas elas criam uma ponte. Transformam o abstrato em algo que pode ser observado e explorado. Oferecem um apoio para que a abstração, depois, possa ser feita com mais segurança. Do ponto de vista teórico acompanhamos de perto as referências [5, 1, 3, 9, 7, 4, 6, 2, 8] e para os mais interessados podem ver com mais detalhes no texto de dissertação intitulado “Animações Matemáticas com *Manim*: do modelo cognitivo à visualização na Educação Básica”

1.2 Por que o Manim?

O Manim (*Mathematical Animation Engine*) é uma biblioteca gratuita e de código aberto para a linguagem Python. Foi criada por Grant Sanderson, o matemático por trás do canal 3Blue1Brown, cujas animações são referência mundial em clareza e precisão visual. O que pouca gente sabe é que, por trás daquele resultado sofisticado, há um princípio simples: no Manim, você escreve em Python o que quer que apareça na tela.

Há, porém, uma característica do Manim que vai além da estética e que é especialmente relevante para o ensino de Matemática: a biblioteca opera sobre objetos matemáticos formalmente definidos, não sobre imagens desses objetos. Quando você define uma circunferência no Manim, você informa o centro e o raio; a ferramenta calcula e desenha com precisão matemática. Quando você plota uma reta, ela é a reta descrita pela equação que você escreveu – não uma aproximação visual. Isso significa que as animações produzidas não são apenas visualmente claras; elas são matematicamente corretas por construção.

Você não precisa se tornar um programador para usar este e-book. Precisa apenas

de disposição para seguir instruções passo a passo. Os códigos apresentados são curtos, comentados linha a linha e pensados para quem nunca programou. Não estamos propondo que você domine Python. Estamos propondo que você tenha, em mãos, uma ferramenta para produzir materiais visuais fundamentados para os conteúdos que você ensina.

1.3 Os descritores trabalhados neste e-book

O SAEB e os descritores do currículo

O Sistema de Avaliação da Educação Básica (SAEB) organiza as habilidades matemáticas esperadas dos estudantes em **descritores** – elementos que detalham competências específicas avaliáveis em cada etapa da escolaridade. As Matrizes de Referência do SAEB não abrangem todo o currículo, mas definem o que é possível e relevante avaliar em larga escala, funcionando como um mapa das habilidades centrais de cada etapa.

Este e-book trabalha onze descritores distribuídos por diferentes temas do currículo dos anos finais do Ensino Fundamental. Esses descritores foram selecionados a partir de um critério específico: cada um deles envolve um objeto matemático cuja natureza é dinâmica, transformacional ou processual – ou seja, um objeto que se entende melhor vendo do que lendo. Para cada descritor, foi produzida uma animação com o Manim e um guia de uso em sala de aula.

Os onze descritores e sua organização

Os descritores estão agrupados em duas frentes. A primeira reúne sete descritores de diferentes temas do currículo, cada um representando uma estrutura matemática distinta:

A segunda frente concentra quatro descritores do eixo algébrico do 9º ano, que formam um arco conceitual articulado – da percepção de padrões à interpretação geométrica de sistemas:

Além desses onze descritores, o Capítulo 15 apresenta animações para outros tópicos frequentes no currículo – funções afim e quadrática, Teorema de Pitágoras, juros compostos e probabilidade – demonstrando que a metodologia deste e-book é transferível para qualquer conteúdo que envolva processos e transformações.

Tabela 1.1: Descritores da primeira frente e suas estruturas

Descritor	Habilidade	Estrutura
D1	Localização e movimentação em mapas e planos	Dinâmica / Transformacional
D2	Planificações de sólidos geométricos	Transformacional
D6	Ângulo como giro	Dinâmica
D9	Coordenadas no plano cartesiano	Dinâmica
D16	Números inteiros na reta numérica	Dinâmica
D28	Porcentagem como relação parte-todo	Transformacional
D29	Proporcionalidade direta e inversa	Dinâmica / Relacional

Tabela 1.2: Descritores do eixo algébrico do 9º ano

Descritor	Habilidade	Estrutura
D32	Expressão algébrica de regularidades em sequências	Sequencial
D33	Equações e inequações de 1º grau	Relacional
D34	Sistemas de equações de 1º grau	Relacional
D35	Representação geométrica de sistemas	Dinâmica / Transformacional

1.4 O que você encontrará em cada capítulo de conteúdo

Cada capítulo dedicado a um descritor segue sempre a mesma estrutura, com quatro momentos articulados.

O primeiro é **antes de exibir**: você encontrará uma nota sobre o que seu aluno precisa já saber para que a animação faça sentido, e uma atividade de aquecimento para conduzir oralmente antes de ligar o projetor.

O segundo é **a animação em si**: uma descrição narrativa do que acontece na tela, seguida do código Python comentado linha a linha. Uma caixa de dica técnica acompanha o código, explicando o recurso central do Manim usado naquela animação.

O terceiro é **durante a exibição**: um roteiro com pontos de pausa específicos – o momento exato em que vale pausar o vídeo e a pergunta a fazer antes de continuar.

É nessa pausa que a visualização deixa de ser um evento passivo e se torna um objeto de raciocínio.

O quarto é **depois de ver**: uma atividade de consolidação que transfere o que foi visto para outro registro – papel, caderno, manipulação de material concreto ou resolução de um problema novo. A animação inicia o processo de compreensão; a atividade o consolida.

1.5 O que este e-book não é

Este e-book foi construído a partir de uma fundamentação teórica sobre visualização e aprendizagem matemática, desenvolvida na pesquisa que o originou. Aqui, essa fundamentação aparece de forma prática – nas orientações de uso, nas notas sobre conhecimento prévio e nas sugestões de mediação – sem reproduzir a discussão acadêmica que lhe dá sustentação. Se você se interessa pelo embasamento teórico, a dissertação associada a este produto apresenta, em detalhe, os referenciais que orientaram as decisões de design de cada animação.

Este e-book também não é um curso completo de Python ou de Manim. O foco aqui é o uso pedagógico da ferramenta: produzir animações fundamentadas para conteúdos específicos do currículo, com orientações claras sobre como usá-las em sala de aula. Para quem quiser se aprofundar na programação, o Capítulo 16 indica onde encontrar documentações e tutoriais avançados.

1.6 Uma nota sobre o que a pesquisa diz

Para quem se interessa pelo embasamento, vale registrar que o uso de visualizações no ensino de Matemática é objeto de investigação acadêmica consolidada. Estudos de revisão sistemática confirmam que a visualização externa – aquela que o professor disponibiliza para o aluno – é parte integrante do raciocínio matemático, não mero recurso ilustrativo. Pesquisas específicas sobre o Manim documentam experiências positivas com a criação de animações para o ensino, tanto na Educação Básica quanto no nível universitário, indicando que animações bem planejadas ajudam os estudantes a visualizar o abstrato, a compreender o porquê dos procedimentos e a se engajar mais profundamente com os conceitos.

O que as pesquisas também mostram – e este e-book leva a sério – é que a animação, por si só, não garante a aprendizagem. O que faz diferença é a mediação: o que o professor faz antes, durante e depois da exibição. É por isso que cada capítulo dedica atenção igual ao código e às orientações de uso.

Mas você não precisa ler os artigos para perceber isso em sala. Basta experimentar com atenção ao que acontece com seus alunos.

Como navegar neste e-book

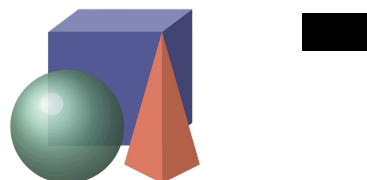
Os primeiros capítulos preparam o terreno: a instalação do ambiente (Capítulo 2), os objetos básicos do Manim (Capítulo 3) e os princípios de design de animações pedagógicas (Capítulo 4). Se você já tem o Manim instalado e alguma familiaridade com a ferramenta, pode ir diretamente ao capítulo do descritor que quer trabalhar – cada capítulo de conteúdo é autossuficiente.

Os capítulos de D1 a D35 (Capítulos 5 a 14) seguem a estrutura dos quatro momentos descrita acima. O Capítulo 15 apresenta animações para tópicos adicionais. O Capítulo 16 reúne orientações para produção, distribuição e continuidade.

Afinal, a Matemática da Educação Básica não precisa ser apenas aquela lista de exercícios no caderno. Ela pode ser também aquela reta que surge no plano, aquele ponto que se desloca, aquele padrão que se revela. Ela pode ser vista. E, quando é vista, pode ser compreendida.

2

Preparando o ambiente



ANTES de criar qualquer animação, precisamos preparar o computador com as ferramentas necessárias. Este capítulo guiará você, passo a passo, na instalação do Python e da biblioteca Manim. Não se preocupe: mesmo que você nunca tenha programado, conseguirá acompanhar. Ao final, você terá rodado sua primeira animação e estará pronto para os próximos capítulos.

2.1 Objetivos do capítulo

Ao final deste capítulo, você será capaz de instalar o Python e o Manim no seu computador, verificar se a instalação funcionou corretamente executando uma animação simples, e se familiarizar com o fluxo básico de trabalho: escrever o código, renderizar e visualizar o resultado.

2.2 Pré-requisitos

Para acompanhar este capítulo, você precisa de um computador com Windows, macOS ou Linux, conexão com a internet para download dos instaladores, e nenhum conhecimento prévio de programação.

2.3 Instalação passo a passo

Instalando o Python

Acesse o site oficial python.org e baixe a versão mais recente do Python (3.8 ou superior). Durante a instalação no Windows, **não se esqueça de marcar a opção** “Add Python to PATH”. No macOS, utilize o instalador padrão; no Linux, use o gerenciador de pacotes da sua distribuição (por exemplo, `sudo apt install python3` no Ubuntu).

Após a instalação, abra o terminal (ou prompt de comando) e digite:

```
1 python --version
```

ou, em alguns sistemas,

```
1 python3 --version
```

Se a instalação estiver correta, a versão do Python será exibida.



Dica

2.1

Considere usar ambientes virtuais (`venv`) para isolar as dependências de cada projeto. Isso evita conflitos entre pacotes e facilita a reprodução do ambiente em outros computadores. Crie com: `python -m venv manim_env` e ative antes de instalar o Manim.

Instalando o Manim (Community Edition)

Figura 2.1: Manim



Fonte: Autoria própria (2026).

No terminal, execute:

```
1 pip install manim
```

ou, se necessário,

```
1 pip3 install manim
```

Aguarde enquanto o pip baixa e instala a biblioteca e suas dependências.

Dependências adicionais (opcionais)

O Manim pode usar o LaTeX para renderizar fórmulas matemáticas com alta qualidade. Se desejar esse recurso, instale uma distribuição LaTeX: no Windows, use o MiKTeX (<https://miktex.org>); no macOS, o MacTeX (<https://tug.org/mactex>); no Linux, o texlive-full via gerenciador de pacotes.

Para a maioria das animações deste e-book, o Manim funciona bem sem LaTeX, utilizando a classe `Text` no lugar de `MathTex`.

2.4 Testando a instalação

Crie uma pasta para seus projetos – por exemplo, `meus_projetos_manim`. Dentro dela, crie um arquivo chamado `teste.py` com o seguinte conteúdo:

```
1 from manim import *
2 class PrimeiraCena(Scene):
3     def construct(self):
4         texto = Text("Olá, professora!")
5         self.play(Write(texto))
6         self.wait(2)
7
```

Código 2.1: Primeira Cena

Para visualizar a animação no YouTube, faça a leitura deste QRcode ou acesse o *link*:



▶ Video 2.1

Primeira Cena

<https://youtu.be/ThzV0Y1SdU4>

Salve o arquivo. No terminal, navegue até a pasta e execute:

```
1 manim -pql teste.py PrimeiraCena
```

Os parâmetros significam: `-p` reproduz automaticamente o vídeo após a renderização; `-ql` usa qualidade baixa, o que torna a renderização muito mais rápida durante os testes.

Se tudo funcionar, uma janela será aberta mostrando o texto sendo “escrito” na tela. Parabéns – você acaba de criar sua primeira animação com Manim!



Dica

2.2

Use a qualidade baixa (`-ql`) durante o desenvolvimento para testes rápidos. Quando a animação estiver finalizada, renderize em alta qualidade com `-qh` para obter vídeos em 1080p. O tempo de renderização aumenta significativamente, mas o resultado final vale a pena.

2.5 Erros comuns e como resolvê-los

Se o terminal mostrar “Comando não encontrado”, o Python não foi adicionado ao PATH – reinstale marcando essa opção. Se aparecer “ModuleNotFoundError: No module named ‘manim’”, a instalação do Manim falhou; tente novamente com `pip install manim` como administrador, se necessário. Se houver problemas com LaTeX, substitua `MathTex` por `Text` ou `Tex` nos exemplos.

2.6 Organizando o fluxo de trabalho

Para manter os projetos organizados, vale seguir algumas convenções simples: crie uma pasta para cada ano ou conteúdo (ex.: `9ano`); dentro dela, coloque os arquivos `.py` com nomes descritivos (ex.: `sequencia_pares.py`); após a renderização, os vídeos serão salvos em `media/videos/...` e podem ser movidos para outra pasta, se preferir.

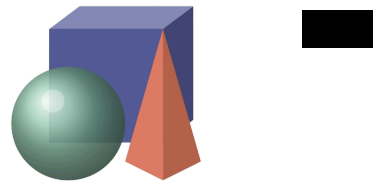
2.7 Desafio para o professor

Modifique o código da primeira animação: altere o texto para “Matemática é movimento!”, mude a cor do texto para azul (`color=BLUE` dentro de `Text`) e renderize novamente para observar a diferença.

Agora que o ambiente está pronto, vamos conhecer os principais objetos e animações do Manim no próximo capítulo.

3

Primeiros passos com objetos para as animações



A GORA que o Manim está instalado e funcionando, é hora de conhecer os blocos de construção fundamentais da biblioteca. Este capítulo apresenta os principais objetos (chamados de `Mobjects`) e as animações mais comuns, que aparecerão repetidamente nos próximos capítulos. Não é preciso memorizar tudo agora – o objetivo é familiarizar-se com os conceitos para que, ao ver os códigos dos descritores, você reconheça os elementos e entenda o que cada parte faz.

3.1 Objetivos do capítulo

Ao final deste capítulo, você conhecerá os principais tipos de objetos do Manim (formas geométricas, textos e expressões matemáticas), saberá posicioná-los na tela, explorará as animações mais comuns (criação, transformação e movimento) e compreenderá como agrupar elementos com `VGroup`.

3.2 O plano cartesiano do Manim

Antes de criar objetos, é importante entender como o Manim organiza o espaço da tela.



Definição

3.1

[Sistema de coordenadas do Manim] O Manim utiliza um sistema de coordenadas cartesianas centrado na tela, onde o centro tem coordenadas $(0, 0, 0)$, o eixo x cresce para a direita (`RIGHT`), o eixo y cresce para cima (`UP`), e o eixo z cresce para fora da tela (`OUT`). A largura padrão da tela é de 14 unidades e a altura é 8 unidades.

Este sistema difere do sistema de pixels tradicional. Cada unidade no Manim corresponde a uma medida relativa que se escala automaticamente com a resolução do vídeo final.



Dica

3.1

Use `ORIGIN` como referência para posicionar objetos no centro da tela. Para visualização durante o desenvolvimento, adicione temporariamente `axes = Axes()` à sua cena para ver os eixos de referência.

Para posicionar objetos, usamos métodos como `shift(UP)` para mover para cima, `shift(RIGHT * 3)` para mover 3 unidades para a direita, `next_to(outro_objeto, DOWN)` para posicionar abaixo de outro objeto, e `to_edge(UP)` para levar o objeto até a borda superior.

3.3 Objetos básicos (Mobjects)



Definição

3.2

[Mobject] `Mobject` (abreviação de “Mathematical Object”) é a classe base de todos os objetos visíveis no Manim. Qualquer elemento que aparece na tela é um `Mobject` ou deriva desta classe.

Todos os `Mobjects` compartilham métodos comuns como `shift()`, `scale()`, `rotate()`, `set_color()` e `move_to()`. Essa uniformidade facilita a manipulação de diferentes tipos de objetos de forma consistente.

3.3.1 Formas geométricas

O Manim oferece uma variedade de formas prontas:

`shift()` move o objeto a partir de sua posição atual; `add()` insere os objetos na cena instantaneamente, sem animação.

Para visualizar a animação no YouTube, faça a leitura deste QRcode ou acesse o *link*:



▶ Video 3.1

Formas geométricas

https://youtu.be/YY86nslg_iM

3.3.2 Texto e expressões matemáticas

Para escrever texto ou fórmulas, usamos:

`Text()` cria texto comum; `MathTex()` cria expressões matemáticas formatadas com LaTeX (observe a dupla barra invertida para caracteres especiais dentro da string Python); `Write()` anima o aparecimento do texto símbolo por símbolo.



Dica

3.2

Para textos em português com acentos, certifique-se de que seu arquivo está salvo em UTF-8. Se encontrar erros de compilação com `MathTex`, verifique se a distribuição LaTeX está instalada corretamente.



▶ Video 3.2

Texto e formulas

```
1 from manim import *
2 class FormasBasicas(Scene):
3     def construct(self):
4         circulo = Circle(
5             color=RED, fill_opacity=0.5
6         )
7         quadrado = Square(
8             color=BLUE, fill_opacity=0.5
9         )
10        triangulo = Triangle(
11            color=GREEN, fill_opacity=0.5
12        )
13        retangulo = Rectangle(
14            width=2, height=1,
15            color=YELLOW, fill_opacity=0.5
16        )
17        linha = Line(LEFT, RIGHT, color=WHITE)
18        ponto = Dot(ORIGIN, color=RED)
19        circulo.shift(LEFT*3 + UP*2)
20        quadrado.shift(RIGHT*3 + UP*2)
21        triangulo.shift(LEFT*3 + DOWN*2)
22        retangulo.shift(RIGHT*3 + DOWN*2)
23        linha.move_to(ORIGIN)
24        ponto.move_to(ORIGIN)
25        self.add(circulo, quadrado,
26                triangulo, retangulo, linha, ponto)
27        self.wait(2)
```

Código 3.1: Formas Geométricas

```

1 from manim import *
2 class TextoEFormulas(Scene):
3     def construct(self):
4         texto = Text("Olá, professora!",
5                       color=BLUE)
6         formula = MathTex(
7             "x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}"
8         )
9         texto.to_edge(UP)
10        formula.next_to(texto, DOWN, buff=1)
11        self.play(Write(texto))
12        self.play(Write(formula))
13        self.wait(2)

```

Código 3.2: Texto e fórmula

<https://youtu.be/CSyd4PloDR4>

3.4 Animações fundamentais



Definição

3.3

[Animação no Manim] Uma animação no Manim é uma transformação gradual de um ou mais `Mobjects` ao longo do tempo. Todas as animações são executadas dentro do método `construct()` de uma `Scene` e são invocadas através do método `self.play()`. O Manim interpola automaticamente entre os estados inicial e final dos objetos, criando movimentos suaves sem necessidade de cálculos manuais de frames intermediários.

3.4.1 Criação e desaparecimento

As animações mais básicas fazem os objetos aparecerem ou sumirem:

```
1 from manim import *
2 class AnimacoesBasicas(Scene):
3     def construct(self):
4         quadrado = Square()
5         circulo = Circle()
6         quadrado.shift(LEFT*3)
7         circulo.shift(RIGHT*3)
8         self.play(Create(quadrado))
9         self.play(FadeIn(circulo))
10        self.wait(1)
11        self.play(FadeOut(quadrado))
12        self.wait(1)
```

Código 3.3: Aparecer e desaparecer

`Create()` desenha a borda do objeto como se estivesse sendo traçada; `FadeIn()` e `FadeOut()` controlam a opacidade; `wait()` pausa a animação por alguns segundos.



▶ Video 3.3

Texto e formulas

<https://youtu.be/bTqoLW2gMEY>

3.4.2 Transformação

Uma das características mais poderosas do Manim é a capacidade de transformar um objeto em outro:

```
1 from manim import *
2 class Transformacao(Scene):
3     def construct(self):
4         quadrado = Square(color=BLUE)
5         circulo = Circle(color=RED)
6         self.play(Create(quadrado))
7         self.wait(0.5)
8         self.play(Transform(quadrado, circulo))
9         self.wait(2)
```

Código 3.4: Transformação de formas

`Transform()` interpola suavemente entre formas, cores e posições.



Dica

3.3

Para que `Transform()` funcione bem, os objetos devem ter estrutura similar (mesmo número de pontos de ancoragem). Se necessário, use `ReplacementTransform()` para substituição completa sem interpolação.



Video 3.4

Texto e formulas

<https://youtu.be/z2yZWkHpe4>

3.4.3 Movimento

Para mover objetos, usamos a sintaxe `objeto.animate.shift()`:

```
1 from manim import *
2 class Movimento(Scene):
3     def construct(self):
4         ponto = Dot(color=RED)
5         self.add(ponto)
6         self.play(ponto.animate.shift(UP*2))
7         self.wait(0.5)
8         self.play(ponto.animate.shift(RIGHT*3))
9         self.wait(0.5)
10        self.play(ponto.animate.shift(
11            DOWN*2 + LEFT*3))
12        self.wait(2)
```

Código 3.5: Movimento



▶ Video 3.5

Movimento

<https://youtu.be/OZSedkQH5tA>

3.5 Posicionamento avançado

3.5.1 Usando next_to

O método `next_to()` é extremamente útil para organizar elementos na tela:

```
1 from manim import *
2 class Posicionamento(Scene):
3     def construct(self):
4         titulo = Text(
5             "Propriedades dos triângulos"
6         ).to_edge(UP)
7         triangulo = Triangle().set_color(GREEN)
8         legenda = Text(
9             "Triângulo equilátero"
10        ).scale(0.5).next_to(triangulo, DOWN)
11        self.add(titulo)
12        self.play(Create(triangulo))
13        self.play(Write(legenda))
14        self.wait(2)
```

Código 3.6: Posicionamento com next_to



▶ Video 3.6

Movimento

<https://youtu.be/WPsYRzdToHc>

3.5.2 Trabalhando com grupos: VGroup

Quando precisamos mover ou animar vários objetos como um conjunto, usamos VGroup:



Definição

3.4

[VGroup] VGroup (Vectorized Group) é um container que agrupa múltiplos Mobjects permitindo que sejam manipulados como um único objeto. Operações aplicadas ao grupo afetam todos os seus membros simultaneamente.

```
1 from manim import *
2 class Agrupamento(Scene):
3     def construct(self):
4         pontos = VGroup(*[
5             Dot(radius=0.1, color=BLUE)
6             for _ in range(5)
7         ])
8         pontos.arrange(RIGHT, buff=0.5)
9         pontos.center()
10        self.play(Create(pontos))
11        self.wait(1)
12        self.play(pontos.animate.shift(UP*2))
13        self.wait(1)
```

Código 3.7: Agrupamento



Video 3.7

Agrupamento

<https://youtu.be/NsOBgAI89nA>

VGroup(*lista) cria um grupo com todos os pontos; arrange(RIGHT, buff=0.5) organiza os elementos horizontalmente com espaçamento de 0.5; animar o grupo move todos os pontos juntos.

3.6 Exemplo integrador

Vamos combinar vários conceitos em uma única cena:

```
1 from manim import *
2 class ExemploIntegrador(Scene):
3     def construct(self):
4         titulo = Text(
5             "Meu primeiro sistema visual"
6         ).to_edge(UP)
7         self.play(Write(titulo))
8         eixos = Axes(
9             x_range=[-3, 3, 1],
10            y_range=[-3, 3, 1],
11            x_length=6, y_length=6
12        )
13        eixos.center().shift(DOWN*0.5)
14        self.play(Create(eixos))
```

Código 3.8: Combinação de objetos I

3.7 Resumo dos comandos apresentados

Os objetos vistos neste capítulo foram Circle, Square, Triangle, Rectangle, Line, Dot, Text, MathTex e Axes. Para posicionamento, usamos shift(), next_to(), to_edge(), arrange() e move_to(). As animações fundamentais são Create, FadeIn, FadeOut, Write, Transform e animate.shift(). Para agrupamento, VGroup.

```
1      grafico = eixos.get_graph(  
2          lambda x: x**2, color=YELLOW  
3      )  
4      rotulo = MathTex("y = x^2").next_to(  
5          grafico, UR, buff=0.2  
6      )  
7      self.play(Create(grafico), Write(rotulo))  
8      ponto = Dot(color=RED)  
9      ponto.move_to(  
10         eixos.coords_to_point(1, 1)  
11     )  
12     self.play(FadeIn(ponto))  
13     self.play(ponto.animate.move_to(  
14         eixos.coords_to_point(2, 4)  
15     ))  
16     self.wait(1)  
17     circulo_destaque = Circle(  
18         color=RED, radius=0.3  
19     ).move_to(ponto)  
20     self.play(Create(circulo_destaque))  
21     self.wait(2)
```

Código 3.9: Combinação de objetos II

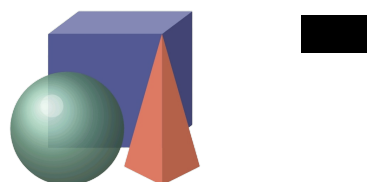
3.8 Desafios para o professor

1. Crie uma cena com um quadrado vermelho no centro, um círculo azul à esquerda e um triângulo verde à direita.
2. Faça o quadrado se transformar em um círculo, depois o círculo se mover para cima e o triângulo desaparecer.
3. Escreva a expressão $a^2 + b^2 = c^2$ e depois transforme-a em $c = \sqrt{a^2 + b^2}$. (Dica: use dois `MathTex` e `Transform`.)
4. Crie um grupo com cinco pontos, organize-os em círculo (pesquise sobre `arrange_in_circle`) e faça o grupo inteiro girar.

Agora que você já conhece os elementos básicos do Manim, está pronto para aplicá-los aos conteúdos específicos do currículo. No próximo capítulo, veremos os princípios de design de animações pedagógicas antes de começar com os descritores.

4

Design de animações



CRiar uma animação pedagogicamente eficaz vai além de dominar a sintaxe do Manim. É preciso planejar a sequência de cenas, controlar o ritmo, escolher cores com critério e pensar na experiência do aluno. Este capítulo oferece orientações práticas para transformar uma ideia matemática em uma animação clara e útil para a sala de aula. Vale lê-lo antes dos capítulos de conteúdo – não porque você precisará aplicar tudo de imediato, mas porque as decisões de design que aparecem nos códigos seguintes fazem mais sentido quando você conhece os princípios que as motivam.

4.1 Planejamento: o storyboard

Antes de escrever qualquer código, vale esboçar no papel a sequência de cenas da animação. Um storyboard não precisa ser elaborado – pode ser uma lista simples com três colunas: o que aparece na tela, em que ordem e quais movimentos ou pausas ocorrem.

Exemplo de storyboard para a sequência dos pares

Cena	Elementos	Ação / Animação
1	Título “Sequência dos pares”	Escrever no topo
2	Termo 2 com índice $n = 1$	Aparecer
3	Termo 4 com índice $n = 2$	Aparecer ao lado
4	Termo 6 com índice $n = 3$	Aparecer
5	Termo 8 com índice $n = 4$	Aparecer
6	Texto “termo = $2 \times n$ ”	Aparecer abaixo
7	Expressão $2n$ em destaque	Surgir no rodapé

Esse esboço já indica claramente o fluxo da animação e evita retrabalho na codificação. Nos capítulos seguintes, cada animação é precedida por um roteiro visual com essa mesma lógica – descrever o que o espectador vê antes de mostrar o código.

4.2 Controle de ritmo e tempo

O ritmo de uma animação é uma variável pedagógica, não apenas estética. Uma animação rápida demais não dá ao aluno tempo para processar o que está vendo; lenta demais perde a atenção. O Manim oferece controle preciso sobre ambos.

Parâmetro `run_time`

Define a duração total de uma animação em segundos:

```
1 self.play(  
2     Create(objeto),  
3     run_time=2    # 2 segundos para construir o objeto  
4 )
```

Código 4.1: Controle de duração com `run_time`

Nos capítulos de conteúdo, você verá que momentos cognitivamente mais exigentes – como a passagem pelo zero na reta numérica ou o fechamento de um sólido em 3D – recebem `run_time` maior que as etapas de configuração inicial. Essa variação não é aleatória: ela dá mais tempo ao aluno precisamente onde o processamento é mais difícil.

Funções de ritmo (`rate_func`)

Controlam a aceleração ao longo do tempo. As mais usadas são `smooth` (padrão) – acelera e desacelera suavemente; `linear` – velocidade constante do início ao fim; `rush_into` – começa rápido e desacelera; `rush_from` – começa devagar e acelera.

```
1 self.play(  
2     objeto.animate.shift(UP),  
3     rate_func=smooth,  
4     run_time=2  
5 )
```

Código 4.2: Controle de aceleração com `rate_func`

Pausas reflexivas

Use `self.wait()` para dar tempo ao aluno de processar o que acabou de aparecer na tela. É nesses segundos de pausa que a visualização interpretativa acontece – quando o aluno para de receber informação e começa a pensar sobre ela. Uma pausa de `wait(0.5)` serve para separar animações rápidas; `wait(2)` é adequada para leitura ou reflexão; `wait()` sem argumento equivale a 1 segundo.



Dica

4.1

Nos capítulos de conteúdo, cada animação tem pontos de pausa marcados com sugestões de perguntas. O `self.wait()` no código corresponde a esses momentos. Quando você pausar o vídeo para fazer uma pergunta, estará fazendo exatamente o que o código planejou.

4.3 Design visual: cor como sinalização

A cor no Manim não é decoração – é sinalização matemática. Sua função pedagógica é orientar a atenção do aluno para o elemento matematicamente relevante naquele momento da animação.

A regra prática é simples: use uma cor por tipo de objeto matemático, consistentemente em todas as suas animações. Se a reta de uma equação é sempre azul e a da outra é sempre verde, o aluno aprende a ler a cor como informação, não como estética. Quando o ponto de interseção aparece em vermelho, ele se destaca imediatamente porque é diferente de tudo que estava na tela.

Paleta sugerida para este e-book

Azul (**BLUE**) para o primeiro elemento, primeira reta, objeto principal. Verde (**GREEN**) para o segundo elemento, objeto complementar. Vermelho (**RED**) para pontos de destaque, soluções e elementos que merecem atenção especial. Amarelo (**YELLOW**) para expressões algébricas finais e resultados que o aluno deve registrar. Cinza (**GRAY**) para elementos de suporte – eixos, grade, referências que não são o foco da cena.

Evite combinações problemáticas para daltônicos (vermelho e verde como distinção principal) e mantenha a paleta consistente em todas as animações do mesmo tema.

Austeridade visual

Uma animação com muitos elementos, cores e movimentos simultâneos pode parecer rica visualmente, mas dificultar a aprendizagem: o aluno gasta energia processando o que não é matematicamente relevante. A regra de ouro é incluir apenas o que tem função matemática explícita na cena. Se um elemento não está ajudando o aluno a entender o conceito, ele está atrapalhando.

Essa austeridade é uma escolha pedagógica deliberada. Nos códigos dos capítulos seguintes, você verá que as animações evitam fundos coloridos, gradientes e elementos decorativos. O que aparece na tela existe porque tem uma razão matemática para estar lá.

4.4 Tipografia nas animações

Para textos curtos e rótulos, use **Text** com fonte sem serifa. Para fórmulas e expressões matemáticas, prefira **MathTex**, que produz a qualidade tipográfica do $\text{\LaTeX}$ diretamente na animação. Evite excesso de tamanhos diferentes: em uma tela de 14 unidades de largura, dois ou três tamanhos são suficientes.

4.5 Exemplo completo: animação do Teorema de Pitágoras

Para praticar o planejamento com storyboard, veja como esses princípios se aplicam a um exemplo concreto.

Storyboard

A animação segue esta sequência: título “Teorema de Pitágoras”; triângulo retângulo com catetos sobre os eixos; quadrado sobre o cateto menor (azul), rotulado a^2 ; quadrado sobre o cateto maior (verde), rotulado b^2 ; quadrado sobre a hipotenusa (amarelo), rotulado c^2 ; fórmula $a^2 + b^2 = c^2$ em destaque.

Código comentado

O triângulo tem vértices em $(-2, -1)$, $(2, -1)$ e $(-2, 2)$, formando um triângulo retângulo de catetos 4 e 3 e hipotenusa 5. Os quadrados são posicionados com arestas coincidentes com os lados do triângulo.

```

1  from manim import *
2
3  class TeoremaPitagoras(Scene):
4      def construct(self):
5
6          titulo = Text("Teorema de Pitágoras").to_edge(UP)
7          self.play(Write(titulo))
8
9          P1 = np.array([-2, -1, 0])
10         P2 = np.array([ 2, -1, 0])
11         P3 = np.array([-2,  2, 0])
12
13         triangulo = Polygon(P1, P2, P3,
14                             color=WHITE, stroke_width=3)
15         self.play(Create(triangulo))
16         self.wait(0.5)
17
18         quad_vert = Polygon(
19             P1, P3,
20             P3 + np.array([-3, 0, 0]),
21             P1 + np.array([-3, 0, 0]),
22             color=BLUE, fill_opacity=0.3
23         )
24         rot_vert = MathTex("a^2", color=BLUE).move_to(
25             np.array([-3.5, 0.5, 0])
26         )
27         self.play(Create(quad_vert), Write(rot_vert))
28         self.wait(0.5)
29
30         quad_hor = Polygon(
31             P1, P2,
32             P2 + np.array([0, -4, 0]),
33             P1 + np.array([0, -4, 0]),
34             color=GREEN, fill_opacity=0.3
35         )
36         rot_hor = MathTex("b^2", color=GREEN).move_to(
37             np.array([0, -3.5, 0])
38         )
39         self.play(Create(quad_hor), Write(rot_hor))
40         self.wait(0.5)
41
42         hip = P3 - P2
43         perp = np.array([hip[1], -hip[0], 0])

```



Observe que os quadrados são construídos usando os vértices do triângulo como âncora, não com `move_to` aproximado. Esse é o princípio dos objetos matemáticos nativos do Manim: as posições são calculadas a partir da geometria real do objeto, garantindo precisão. Sempre que possível, defina posições a partir de vértices ou pontos calculados – não com coordenadas fixadas intuitivamente.

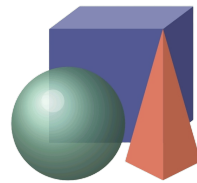
4.6 Desafios para o professor

1. Escolha um conceito do currículo que você ache difícil de explicar com o quadro. Escreva um storyboard de 5 a 7 cenas para uma animação de 1 minuto. Não escreva código ainda – apenas descreva o que o aluno veria na tela.
2. Implemente uma das cenas do seu storyboard usando os objetos do capítulo anterior e os princípios de ritmo e cor deste capítulo.
3. Mostre o resultado para um colega e peça que identifique: qual é o elemento mais importante da cena? Se a resposta for diferente do que você pretendia, revise a sinalização por cor.

Nos próximos capítulos, você verá esses princípios aplicados a cada um dos onze descritores, com animações prontas e orientações para uso em sala de aula. Começamos pelo D1 – localização e movimentação em mapas.

5

Criando animações com auxílio de ferramentas de IA



HÁ uma distância real entre imaginar uma animação e ter o código que a realiza. Você sabe o que quer mostrar: o ponto que cruza o zero, as duas retas que se encontram, a face do cubo que dobra. Mas traduzir isso em Python, linha a linha, leva tempo – e tempo é exatamente o que professores têm pouco.

Nos últimos anos, ferramentas de inteligência artificial passaram a escrever código de programação com uma competência que já é útil na prática. Não são perfeitas. Erram, inventam métodos que não existem, às vezes precisam de mais de uma tentativa. Mas conseguem pegar uma descrição em palavras – o que deve aparecer na tela, em que ordem, com que cores – e transformá-la em código Manim que roda.

Este capítulo mostra como usar essas ferramentas. Não é preciso ter lido os capítulos anteriores para acompanhar o essencial – se você chegou aqui diretamente, vai encontrar o que precisa para começar. Mas se você já percorreu os capítulos ?? e ??, vai conseguir avaliar e ajustar o que a ferramenta produz com muito mais segurança. A leitura do código gerado é mais fácil quando você já reconhece os objetos.

5.1 O que a ferramenta faz – e o que ela não faz

Uma ferramenta de IA consegue, dado um pedido bem descrito, escrever um código Manim que roda. Ela conhece a sintaxe, sabe os nomes dos objetos e consegue montar a sequência

de animações se você explicar o que quer que aconteça na tela. Isso já elimina a parte mais trabalhosa do processo: escrever as linhas certas, na ordem certa, sem erros de digitação.

O que ela não faz é mais importante de entender: ela não sabe se a animação está matematicamente correta. Ela pode produzir um código impecável que mostra o ponto chegando no lugar errado – e não vai perceber isso. Ela não conhece a sua turma, não sabe o que já foi trabalhado, não percebe quando uma animação vai rápido demais para o ritmo daquela aula específica. Esses julgamentos são seus.

Por isso, a ordem das decisões não muda: primeiro você define o que a animação precisa mostrar e por quê. A ferramenta entra depois, para ajudar a escrever o código que realiza o que você já pensou. Ela é rápida na execução; você é insubstituível no julgamento.

5.2 Como descrever o que você quer

O que a ferramenta entrega depende diretamente de como você descreve o pedido. Um pedido vago produz uma animação genérica. Um pedido que descreve o que aparece na tela, em que ordem e com que cores produz um ponto de partida muito mais aproveitável.

Não é preciso saber programar para fazer um bom pedido. Você descreve a animação como descreveria um vídeo que imaginou: o que aparece primeiro, o que se move, para onde, o que surge depois, onde termina. A ferramenta cuida da sintaxe.

Há três informações que fazem diferença em qualquer pedido:

- **A versão da biblioteca.** Escreva sempre “*Manim Community Edition (pacote manim, não manimlib)*”. Existe uma versão mais antiga do Manim, com sintaxe incompatível, e a ferramenta pode confundir as duas se não for orientada.
- **A sequência do que acontece na tela.** Descreva em ordem cronológica – o que aparece primeiro, o que se move, o que surge depois, quando há pausa. Isso se traduz diretamente em chamadas `self.play()` e `self.wait()`, que são os comandos que controlam o ritmo da animação.
- **O formato do código.** Peça o código completo, com a importação e a definição da classe. Sem isso, a ferramenta às vezes entrega apenas fragmentos que não funcionam sozinhos.

O storyboard do Capítulo ?? – aquela lista simples com o que aparece na tela, em que ordem e com quais movimentos – é exatamente o formato que a ferramenta melhor aproveita. Se você já tem o storyboard escrito, pode usá-lo quase diretamente como pedido.



[Pedido para a animação do D16 – números inteiros na reta]

Você é especialista em Manim Community Edition (pacote manim, não manimlib). Quero uma animação que mostre a adição $3 + (-5)$ em uma reta numérica de -6 a 6 . Um ponto amarelo começa no zero. Primeiro, o texto “+3” aparece em verde acima da reta e o ponto se desloca três unidades para a direita (`run_time=2`). Em seguida, o texto “-5” aparece em vermelho e o ponto recua cinco unidades para a esquerda, passando pelo zero devagar (`run_time=3`). Ao final, a expressão $3 + (-5) = -2$ aparece abaixo da reta. Inclua `self.wait(1)` depois de cada movimento e antes do resultado. Forneça o código completo, com importação e classe que herda de `Scene`.

Note que o pedido não menciona nenhum método específico do Manim. Não fala em `animate.move_to`, não fala em `MathTex`, não fala em `n2p`. Você descreve o que o aluno vai ver; a ferramenta resolve como produzir isso em código.

5.3 Do pedido à animação: o caminho completo

Vamos percorrer o processo com o exemplo do D16, o mesmo descritor tratado no Capítulo 11. A ideia é mostrar o que acontece entre o momento em que você descreve o pedido e o momento em que a animação toca na tela.

Passo 1 – Escreva o pedido

Use o modelo da seção anterior. Quanto mais específico, melhor. Se você já tem um storyboard esboçado, use-o como base.

Passo 2 – Cole o código num arquivo `.py`

A ferramenta retorna um bloco de código. Copie tudo e cole num arquivo novo – por exemplo, `d16_inteiros.py`. Não tente executar diretamente no navegador da ferramenta; a mensagem de erro que aparece no terminal do seu computador é muito mais informativa do que qualquer aviso da interface.

A seguir está um exemplo do tipo de código que uma ferramenta bem orientada produz para o pedido do D16:

```
1 from manim import *
2
3 class InteirosNaReta(Scene):
4     def construct(self):
5         reta = NumberLine(
6             x_range=[-6, 6, 1],
7             length=12,
8             include_numbers=True,
9             color=GRAY,
10        )
11        self.add(reta)
12
13        ponto = Dot(reta.n2p(0), color=YELLOW, radius=0.15)
14        self.add(ponto)
15        self.wait(1)
16
17        label_pos = MathTex("+3", color=GREEN).next_to(reta, UP)
18        self.play(Write(label_pos))
19        self.play(ponto.animate.move_to(reta.n2p(3)), run_time=2)
20        self.wait(1)
21
22        label_neg = MathTex("-5", color=RED).next_to(reta, UP)
23        self.play(Transform(label_pos, label_neg))
24        self.play(ponto.animate.move_to(reta.n2p(-2)), run_time=3)
25        self.wait(1)
26
27        resultado = MathTex(
28            "3 + (-5) = -2"
29        ).next_to(reta, DOWN)
30        self.play(Write(resultado))
31        self.wait(2)
```

Passo 3 – Teste em qualidade baixa

No terminal, dentro da pasta onde está o arquivo, execute:

```
manim -pql d16_inteiros.py InteirosNaReta
```

A flag `-ql` renderiza rápido – a animação fica em baixa resolução, mas o essencial é verificar se o código funciona e se o resultado é o que você pretendia. Se uma janela abriu e o vídeo tocou, o código está correto. Se apareceu uma mensagem de erro, a seção seguinte orienta o que fazer.

Passo 4 – Avalie o que apareceu na tela

Este é o passo que a ferramenta não pode fazer por você. O ponto cruza o zero durante o segundo deslocamento? As cores fazem sentido – verde para positivo, vermelho para negativo? A velocidade permite que o aluno acompanhe o raciocínio? A expressão final aparece no momento certo?

Se algo não está como você queria, há dois caminhos: ajustar o código diretamente – o que os capítulos anteriores ensinaram a fazer – ou descrever o problema para a ferramenta e pedir a correção. Ambos funcionam; o segundo é mais rápido quando o erro não está claro.

Passo 5 – Renderize em alta qualidade

Quando o resultado estiver bom, substitua `-ql` por `-qh` para obter o vídeo em 1080p:

```
manim -pqh d16_inteiros.py InteirosNaReta
```

Esse é o arquivo que vai para o projetor ou para a plataforma de vídeo.



Dica

5.1

Guarde o texto do pedido que você escreveu junto com o arquivo `.py`. Se precisar adaptar a animação no futuro – para outro exemplo numérico, para outra turma – é muito mais rápido ajustar o pedido e gerar novamente do que reescrever o código do zero.

5.4 Quando o código não funciona

Acontece. A ferramenta às vezes inventa um método que não existe, esquece a importação ou posiciona um objeto fora da área visível da cena. Não é motivo para desistir – é parte do processo, e tem solução.

A mensagem de erro no terminal sempre indica onde está o problema. Leia a última linha antes de qualquer outra coisa: ela diz o que falhou e em que linha do código. Com essa informação, há três caminhos.

1. **Você reconhece o erro.** Importação ausente, indentação quebrada, nome de método ligeiramente errado – esses são os erros mais comuns e os mais simples de corrigir diretamente no arquivo. O Capítulo ?? cobre os casos mais frequentes.
2. **Você não reconhece o erro.** Copie a mensagem completa do terminal e cole de volta para a ferramenta, junto com o código que ela gerou. Basta escrever: *“Ao rodar este código, apareceu este erro. Pode corrigir?”* As ferramentas atuais conseguem se autocorrigir com frequência quando recebem a mensagem de erro como contexto.
3. **O código roda, mas a animação não faz o que você queria.** Descreva o que apareceu na tela e o que deveria aparecer. Por exemplo: *“O ponto se deslocou para a direita nos dois movimentos, mas o segundo deveria ir para a esquerda.”* Esse tipo de ajuste funciona bem.

A ferramenta não tem como saber se a animação está matematicamente correta. Um código pode rodar sem erros e mostrar o ponto chegando no lugar errado – e isso não vai gerar nenhum aviso. Por isso, sempre assista ao resultado com os olhos de quem conhece o conteúdo, não apenas de quem quer que o vídeo toque.

5.5 Onde rodar o código

Se você instalou o Manim no Capítulo ??, o fluxo já descrito funciona: arquivo `.py`, terminal, `manim -pql`. Mas há situações – o computador da escola, uma máquina sem permissão de instalação, um colega que quer testar sem configurar o ambiente – em que instalar o Manim não é uma opção. Nesses casos, o Google Colab resolve.

Google Colab – sem instalação, direto no navegador

O Google Colab (colab.research.google.com) é um ambiente Python que funciona no navegador, sem nenhuma instalação prévia. Você precisa de uma conta Google e de conexão com a internet.

1. Acesse o Colab e crie um novo notebook.
2. Na primeira célula, instale o Manim:

```
1 !pip install manim
```

Aguarde a instalação terminar – isso leva alguns minutos e precisa ser feito apenas uma vez por sessão.

3. Cole o código gerado em uma nova célula e salve o arquivo. Dê um nome descritivo, como `d16_inteiros.py`.
4. Para renderizar, execute em outra célula:

```
1 !manim -pql d16_inteiros.py InteirosNaReta
```

5. O Manim salva o vídeo numa pasta dentro do ambiente. Para exibi-lo diretamente no Colab, execute:

```
1 import glob
2 from IPython.display import Video
3
4 # Localiza o vídeo gerado automaticamente
5 arquivos = glob.glob("media/videos/**/*.mp4", recursive=True)
6 Video(arquivos[0], embed=True)
```

Esse trecho localiza o vídeo independentemente do nome da pasta que o Manim gera internamente, o que evita o erro mais comum no Colab: tentar abrir um arquivo com o caminho errado.



Se a instalação travar ou o ambiente desconectar, basta reabrir o notebook e rodar a célula de instalação novamente. O Google Colab reinicia o ambiente periodicamente – isso é normal e não significa que algo está errado.

VS Code com extensão Manim Preview

Se você usa o VS Code, a extensão *Manim Preview* permite renderizar a cena ativa diretamente do editor, sem abrir o terminal. Com o arquivo `.py` aberto, pressione **Ctrl+Shift+P** e selecione **Manim: Render scene**. Para quem produz animações com frequência, essa integração economiza bastante tempo.

5.6 Cada ferramenta tem seu jeito

Não existe uma única ferramenta de IA generativa – há várias, e elas se comportam de formas ligeiramente diferentes quando o assunto é código Manim. Não é preciso testar todas; qualquer uma das mais usadas funciona. Mas conhecer as diferenças evita surpresas.

[ChatGPT (OpenAI)] É a ferramenta mais experimentada por quem trabalha com código Manim. Com um pedido detalhado e a menção explícita a “*Community Edition*”, produz código consistente na maior parte das vezes. Em animações longas, pode perder o fio do pedido no meio da geração – nesses casos, vale dividir a animação em partes menores e gerar cada uma separadamente.

[Claude (Anthropic)] Lida bem com pedidos longos e produz código bem comentado, o que facilita a leitura e os ajustes. Aceita arquivos como referência: se você colar o código de um capítulo inteiro antes de fazer o pedido, a ferramenta pode usá-lo para manter coerência com o que você já tem. Às vezes entrega o código sem a linha de importação – vale conferir se o arquivo está completo antes de rodar.

[Gemini (Google)] Entende bem pedidos em português e costuma gerar código funcional. O erro mais frequente é confundir a versão atual do Manim com a versão original, que tem sintaxe diferente – e o terminal indica isso logo na primeira linha. Especificar “*Manim Community Edition, pacote manim, não manimlib*” resolve o problema na maior parte das vezes.

Em todas elas, uma frase de abertura resolve os dois erros mais comuns – versão errada da biblioteca e código incompleto:

Você é especialista em Manim Community Edition (pacote manim, não manimlib). Forneça o código completo, com importação e classe que herda de Scene.

5.7 Comunidade, referências e o que fazer com as animações

O ecossistema em torno do Manim cresceu bastante desde que a versão comunitária foi lançada. Para quem quiser ir além dos exemplos deste e-book – adaptar uma animação, explorar recursos novos, resolver um erro que a ferramenta não conseguiu corrigir –, há bons pontos de partida.

O repositório oficial [ManimCommunity/manim](https://github.com/ManimCommunity/manim) reúne a documentação completa, exemplos verificados e um espaço para reportar problemas. O [awesome-manim](https://github.com/ManimCommunity/awesome-manim) é uma coleção organizada de tutoriais e projetos feitos com a biblioteca – boa fonte quando você quer ver o que é possível fazer além do básico. O servidor Discord da comunidade (discord.gg/manim) é ativo e acolhedor com iniciantes: colar o código e a mensagem de erro lá costuma resultar em resposta rápida.

As animações que você produzir são arquivos `.mp4` comuns e podem ser usados em qualquer situação que um vídeo resolve. No projetor da sala, a forma mais direta. No YouTube, o caminho mais simples para que os alunos acessem fora da aula – organize por conteúdo ou por turma. No WhatsApp ou Telegram, vídeos curtos funcionam bem quando acompanhados de uma frase que contextualiza: “*Esse vídeo mostra por que $3 + (-5)$ dá negativo mesmo começando num número positivo.*” Sem contexto, o vídeo vira só mais um arquivo no grupo.

Ao publicar, o mesmo princípio do Capítulo ?? se aplica: cada elemento da animação existe por um motivo. Efeitos extras e transições decorativas que não carregam informação matemática tornam o vídeo mais confuso, não mais interessante. A animação serve para mostrar o conceito – não para impressionar.

5.8 Considerações finais

Usar uma ferramenta de IA para escrever código Manim é uma escolha prática, não uma concessão. Professores têm tempo limitado, e escrever cada animação linha a linha pode ser inviável no ritmo da sala de aula. Se a ferramenta escreve o rascunho e você revisa, testa e ajusta, o resultado é o mesmo – a animação que você pensou, para a turma que você conhece.

Dois pontos valem ser ditos com clareza. O primeiro: quanto mais você conhece o Manim, mais rápido percebe quando o código gerado está errado e mais fácil é corrigi-lo.

Os capítulos anteriores não foram preparação burocrática – foram o que permite que o seu julgamento seja confiável. O segundo: a ferramenta não sabe matemática. Nunca assume que o resultado está correto sem assistir ao vídeo e verificar.

Fora isso, experimente. Pegue um descritor que você ainda não animou, descreva o que você quer ver na tela e teste o que a ferramenta produz. A distância entre a ideia e a animação nunca foi menor.



Desafio

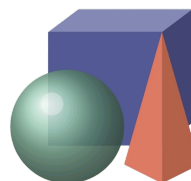
5.1

Escolha um descritor deste e-book que você ainda não animou – D28 (porcentagem) ou D29 (proporcionalidade) são bons pontos de partida. Escreva um pedido seguindo o modelo da Seção 5.2 e gere o código com a ferramenta de sua preferência. Rode no terminal ou no Colab e assista ao resultado. Compare com o código do capítulo correspondente: o que a ferramenta fez diferente? O que está matematicamente correto? O que precisaria ser ajustado para servir à sua turma? Faça os ajustes, renderize a versão final e, se quiser, compartilhe com um colega.

Os capítulos seguintes apresentam cada descritor como um ponto de partida – não como receita fechada. Você pode usar os códigos como estão, adaptá-los com a ferramenta, ou criar versões novas a partir do zero. O material é seu; a ferramenta é só mais um caminho para chegar lá.

6

Localização e movimentação em mapas: D1



LOCALIZAR um ponto num mapa e descrever como chegar de um lugar a outro são habilidades que os alunos exercitam desde cedo no cotidiano. No entanto, quando essa mesma habilidade aparece no plano cartesiano com coordenadas e convenções formais, muitos travam. O problema não está na dificuldade do conceito – está na representação. Uma seta numa figura estática indica que houve movimento, mas não mostra o movimento. A animação substitui a convenção pela experiência: o aluno vê o deslocamento acontecer, com direção, sentido e distância percorrida construídos na tela enquanto ele observa.

6.1 Por que animar este conteúdo?

O objeto matemático central do D1 não é um ponto fixo no plano – é o processo de mudança de posição ao longo do tempo. Representações estáticas capturam o estado inicial e o estado final de um deslocamento, mas suprimem exatamente o que define o conceito: a correspondência entre direção, sentido e variação de posição como processo contínuo.

Um aluno que aprende deslocamento apenas por setas em figuras precisa reconstruir mentalmente o movimento a partir dessas convenções. Muitos não conseguem. A animação dispensa essa inferência ao tornar o deslocamento diretamente observável: o ponto parte, percorre um trajeto visível e chega. O que antes precisava ser imaginado agora pode ser visto.

6.2 O que a animação mostra

A animação abre com um plano cartesiano de grade discreta, com eixos numerados e opacidade reduzida para não competir com o elemento principal. Um ponto azul aparece na posição inicial $(2, 3)$ e permanece estático por dois segundos – tempo para que o aluno registre onde o ponto está antes de qualquer movimento.

Em seguida, o ponto se desloca suavemente para norte, percorrendo três unidades no eixo vertical. Durante o movimento, um rastro azul acompanha o percurso, tornando a trajetória um objeto visível e persistente na tela mesmo depois que o ponto chegou ao destino. O deslocamento e sua medida são exibidos ao lado. A animação pausa dois segundos antes de introduzir o segundo deslocamento: o ponto parte da nova posição $(2, 6)$ e se move para leste, percorrendo quatro unidades no eixo horizontal.

A cena encerra com os dois rastros visíveis simultaneamente, as medidas anotadas e os eixos identificados. O aluno pode ler o percurso completo como sequência de dois deslocamentos distintos, cada um com direção e comprimento próprios.

6.3 Antes de exibir

O que o aluno precisa saber

Esta animação pressupõe que o aluno já sabe identificar coordenadas no plano cartesiano: localizar um ponto dado o par ordenado e escrever as coordenadas de um ponto marcado no plano. Sem essa base, a atenção vai para os elementos do plano em vez de para o deslocamento – o aluno tentará entender o sistema de coordenadas enquanto deveria estar acompanhando o movimento.

Se a turma ainda não domina a leitura de coordenadas com fluência, vale dedicar uma aula prévia apenas ao plano estático – localizar pontos, ler coordenadas, identificar quadrantes – antes de introduzir qualquer deslocamento.

Atividade de aquecimento

Antes de abrir o computador, desenhe no quadro o plano cartesiano com um ponto marcado em $(2, 3)$. Pergunte à turma:

- “Quais são as coordenadas desse ponto?”
- “Se esse ponto se deslocar 3 unidades para cima, onde vai parar?”
- “E se depois se deslocar 4 unidades para a direita?”

Deixe os alunos responderem e marcarem os pontos no quadro antes de exibir a animação. A confrontação entre a previsão e o que a animação mostra é pedagogicamente mais poderosa do que a observação passiva.

6.4 O código comentado

A decisão central desta animação é o `TracedPath`: ele registra e exibe continuamente a posição do ponto durante o movimento, produzindo o rastro que torna a trajetória um objeto visível e não apenas um resultado. Sem ele, o aluno vê o ponto chegar; com ele, o aluno vê o ponto percorrer.

```

1  from manim import *
2
3  class DeslocamentoNoPlano(Scene):
4      def construct(self):
5
6          # Plano cartesiano com grade de baixa opacidade
7          plano = NumberPlane(
8              x_range=[-1, 8, 1],
9              y_range=[-1, 8, 1],
10             background_line_style={"stroke_opacity": 0.3}
11         )
12         self.add(plano)
13
14         # Ponto de partida: posição inicial (2, 3)
15         ponto = Dot(plano.c2p(2, 3), color=BLUE, radius=0.12)
16         self.add(ponto)
17         self.wait(2)    # pausa para registrar a posição inicial
18
19         # TracedPath: segue o ponto e deixa rastro visível
20         rastro = TracedPath(
21             ponto.get_center,
22             stroke_color=BLUE,
23             stroke_width=3
24         )
25         self.add(rastro)
26
27         # Primeiro deslocamento: (2,3) (2,6)
28         self.play(
29             ponto.animate.move_to(plano.c2p(2, 6)),
30             run_time=2
31         )
32
33         med1 = MathTex(r"\uparrow", 3, color=BLUE).scale(0.8)
34         med1.next_to(plano.c2p(2, 4.5), RIGHT, buff=0.2)
35         self.play(FadeIn(med1))
36         self.wait(1.5) # ponto de pausa pedagógico
37
38         # Segundo deslocamento: (2,6) (6,6)
39         self.play(
40             ponto.animate.move_to(plano.c2p(6, 6)),
41             run_time=2
42         )
43

```



O `TracedPath` recebe `ponto.get_center` – sem parênteses. Isso passa a *função* que retorna o centro do ponto, não o valor fixo. É essa distinção que permite ao `TracedPath` acompanhar o ponto em tempo real durante a animação. Com parênteses, você passaria apenas a posição inicial e o rastro não se moveria.

6.5 Durante a exibição

Pausa 1 – após o ponto aparecer, antes do primeiro movimento

Pause assim que o ponto estiver parado em (2, 3). Pergunte: “Alguém consegue dizer as coordenadas desse ponto?” Se a atividade de aquecimento foi feita, os alunos já sabem a resposta – essa confirmação reforça a conexão entre o plano que eles conhecem e o que está na animação.

Pausa 2 – após o primeiro deslocamento, antes do segundo

Pause quando o ponto estiver em (2, 6) com o rastro vertical visível. Pergunte: “Para onde o ponto se moveu? Quantas unidades? Em que direção? Quais são as coordenadas agora?” Anote as respostas no quadro.

Pausa 3 – antes de revelar a posição final

Pause antes de exibir o rótulo (6, 6). Pergunte: “Se eu pedir para o ponto voltar à posição original (2, 3) pelo caminho mais curto, qual seria o deslocamento necessário?” Essa pergunta introduz o conceito de deslocamento inverso sem precisar de definição formal.

6.6 Depois de ver

Atividade de consolidação

Peça que os alunos descrevam por escrito o percurso completo que a animação mostrou, usando linguagem natural: posição inicial, primeiro deslocamento (direção, sentido, número de unidades), posição intermediária, segundo deslocamento, posição final. Em seguida, peça que calculem a distância total percorrida pelo ponto.

A verbalização do percurso é o momento em que a visualização se consolida e se torna avaliável. Um aluno que viu a animação mas não consegue descrevê-la ainda não construiu o conceito – apenas assistiu a um vídeo.

Variação do código

Modifique a animação para mostrar um percurso com três deslocamentos, usando cores diferentes para cada trecho:

```
1  # Após o segundo deslocamento, acrescente:
2
3  rastro.clear_updaters()
4  rastro2 = TracedPath(
5      ponto.get_center,
6      stroke_color=GREEN,
7      stroke_width=3
8  )
9  self.add(rastro2)
10
11 # Terceiro deslocamento: 2 unidades para baixo
12 self.play(
13     ponto.animate.move_to(plano.c2p(6, 4)),
14     run_time=1.5
15 )
16 self.wait(1)
```

Código 6.2: D1 – Variação com terceiro deslocamento em cor diferente

6.7 Desafios para o professor

1. Crie uma animação que mostre um percurso de quatro deslocamentos, formando um caminho fechado que retorna ao ponto de partida.
2. Adapte a animação para usar um mapa estilizado em vez do plano cartesiano abstrato: substitua os eixos por ruas ou referências geográficas simples desenhadas com `Line` e `Text`.
3. Crie uma segunda versão da animação que exiba apenas o estado inicial e o estado final, sem o rastro – e mostre as duas versões para a turma. Peça que comparem: o que a versão com rastro comunica que a sem rastro não comunica?

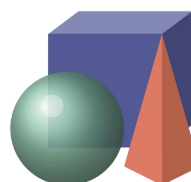


▶ **Video 6.1**

D1 – Localização e movimentação no plano cartesiano

https://youtu.be/INSERIR_URL_D1

Planificações de sólidos geométricos: D2



ALGUÉM que nunca abriu uma caixa de papelão conhece intuitivamente o que é uma planificação – mas raramente consegue fazer a operação inversa: olhar para os seis quadrados recortados numa folha e imaginar que, dobrados, formariam um cubo. É essa operação, a passagem entre o plano e o espaço, que o descritor D2 exige dos alunos. E é exatamente essa operação que nenhuma figura estática consegue mostrar de forma direta. Por isso escolhemos animá-la.

7.1 Por que animar este conteúdo?

Uma planificação no livro didático mostra um estado: o sólido desmontado. O cubo montado mostra outro estado. O que conecta os dois é o dobramento – um processo de transformação que precisa ser imaginado internamente por cada aluno. Alguns conseguem; muitos não.

A animação mostra o processo diretamente: as faces da planificação se dobram progressivamente, uma a uma, até constituir o sólido. O aluno não precisa mais imaginar – ele pode ver. E como cada face recebe uma cor diferente, é possível rastrear exatamente para onde cada pedaço do plano foi parar no espaço. E como cada face é exibida numa cor diferente, o aluno pode rastrear onde cada pedaço do plano foi parar no espaço.

7.2 Uma nota sobre cenas 3D no Manim

Esta é a primeira animação deste e-book que usa `ThreeDScene` em vez de `Scene`. A diferença é simples: `ThreeDScene` tem câmera com profundidade, o que permite mostrar objetos tridimensionais com perspectiva.

O comando `set_camera_orientation` define o ângulo de visão em dois parâmetros:

- `phi` – inclinação vertical da câmera. `phi=60*DEGREES` posiciona a câmera levemente acima do objeto, dando a sensação de ver de cima.
- `theta` – rotação horizontal. `theta=-45*DEGREES` posiciona a câmera no canto, revelando três faces do cubo simultaneamente.

Não é necessário memorizar esses valores agora. Use os do exemplo e, se quiser experimentar outro ângulo de visão, altere o `phi` ou o `theta` e compile para ver o resultado.

7.3 O que a animação mostra

A animação abre com a planificação do cubo disposta no plano horizontal: seis quadrados organizados em cruz, cada um numa cor diferente. A câmera está posicionada em ângulo levemente acima e de canto para que o dobramento em três dimensões fique visível desde o início.

A cena pausa brevemente para que o aluno examine a figura plana e associe cada cor a uma posição. Em seguida, as faces laterais começam a se dobrar progressivamente em direção ao centro, uma a uma. A câmera realiza uma leve rotação para revelar a dimensão emergente à medida que as faces se erguem. A face superior se fecha por último, completando o cubo. A animação pausa com o sólido completo, permitindo que o aluno visualize a correspondência entre as faces coloridas do cubo e os quadrados coloridos da planificação inicial.

7.4 Antes de exhibir

O que o aluno precisa saber

Esta animação pressupõe que o aluno conhece as características das figuras planas que compõem a planificação do cubo – especificamente, que um cubo tem seis faces quadradas iguais. Também é útil que o aluno tenha alguma noção de representação de sólidos, mesmo que incipiente.

Turmas sem nenhum contato anterior com representações tridimensionais podem ter dificuldade em interpretar o movimento de câmera como revelação de profundidade. Um

aluno que nunca viu uma representação de sólido em perspectiva pode confundir a rotação da câmera com a rotação do objeto. Nesses casos, vale apresentar brevemente uma imagem estática do cubo em perspectiva antes de exibir a animação.

Atividade de aquecimento

Se possível, leve para a aula uma caixa de papelão pequena – ou use qualquer embalagem cúbica disponível. Abra a caixa cuidadosamente até planificá-la e mostre as duas formas para a turma. Pergunte:

- “Quantas faces tem este sólido?”
- “Se eu colorir cada face de uma cor diferente antes de abrir a caixa, o que acontece quando eu planificar?”
- “Alguém consegue dobrar essa planificação mentalmente e dizer qual face ficaria em cima?”

Depois, distribua uma folha com a planificação do cubo impressa e peça que os alunos tentem recortar e dobrar antes de ver a animação. A experiência cinestésica do dobramento com as mãos cria a base para que a animação faça sentido.

7.5 O código comentado

O código está dividido em duas partes. A primeira prepara a cena com a planificação e a câmera 3D. A segunda anima o dobramento de cada face.

```
1 from manim import *
2
3 class PlanificacaoCubo(ThreeDScene):
4     def construct(self):
5
6         # Câmera 3D: phi=60 graus acima, theta=-45 graus de canto
7         # Esses valores mostram três faces do cubo simultaneamente
8         self.set_camera_orientation(
9             phi=60 * DEGREES,
10            theta=-45 * DEGREES
11        )
12
13        # Face base: quadrado azul no centro do plano horizontal
14        base = Square(side_length=2, color=BLUE, fill_opacity=0.6)
15        base.set_fill(BLUE)
16
17        # Face frontal: dobra 90° em torno da aresta inferior da base
18        frontal = Square(side_length=2, color=RED, fill_opacity=0.6)
19        frontal.set_fill(RED)
20        frontal.next_to(base, DOWN, buff=0)
21
22        # Face traseira: dobra em torno da aresta superior
23        traseira = Square(side_length=2, color=GREEN, fill_opacity=0.6)
24        traseira.set_fill(GREEN)
25        traseira.next_to(base, UP, buff=0)
26
27        # Face esquerda: dobra em torno da aresta esquerda
28        esquerda = Square(side_length=2, color=ORANGE, fill_opacity=0.6)
29        esquerda.set_fill(ORANGE)
30        esquerda.next_to(base, LEFT, buff=0)
31
32        # Face direita: dobra em torno da aresta direita
33        direita = Square(side_length=2, color=PURPLE, fill_opacity=0.6)
34        direita.set_fill(PURPLE)
35        direita.next_to(base, RIGHT, buff=0)
36
37        # Face superior: dobra sobre a face frontal
38        superior = Square(side_length=2, color=YELLOW, fill_opacity=0.6)
39        superior.set_fill(YELLOW)
40        superior.next_to(frontal, DOWN, buff=0)
41
42        # Mostrar a planificação completa
43        self.add(base, frontal, traseira, esquerda, direita, superior)
```

```
1      # Dobramento das faces laterais
2      # about_point define a aresta de articulação
3      # - não a aresta do quadrado, mas o ponto da base adjacente
4
5      # Face frontal: dobra para cima (eixo RIGHT, ponto inferior da
6      ↪ base)
7      self.play(
8          Rotate(
9              frontal,
10             angle=PI / 2,
11             axis=RIGHT,
12             about_point=base.get_bottom(),
13             run_time=2
14         )
15     )
16     self.wait(0.5)
17
18     # Face traseira: dobra para frente (eixo RIGHT invertido)
19     self.play(
20         Rotate(
21             traseira,
22             angle=-PI / 2,
23             axis=RIGHT,
24             about_point=base.get_top(),
25             run_time=2
26         )
27     )
28     self.wait(0.5)
29
30     # Face esquerda: dobra para dentro (eixo UP)
31     self.play(
32         Rotate(
33             esquerda,
34             angle=PI / 2,
35             axis=UP,
36             about_point=base.get_left(),
37             run_time=2
38         )
39     )
40     self.wait(0.5)
41
42     # Face direita: dobra para dentro (eixo UP invertido)
```



O parâmetro `about_point` é o que garante que o dobramento seja geometricamente correto. Ele define o ponto fixo em torno do qual a face gira – que é sempre a aresta compartilhada entre a face que dobra e a face que permanece estática. Se você alterar `about_point` para qualquer outro ponto, o dobramento parecerá correto visualmente mas deixará lacunas ou sobreposições entre as faces.

7.6 Durante a exibição

Pausa 1 – com a planificação completa na tela

Pause antes de qualquer dobramento. Pergunte: “Quantas faces estão na tela? Quantas faces tem um cubo? São o mesmo número?” Peça que os alunos identifiquem qual quadrado vai ser o topo, qual vai ser o fundo, quais vão ser as laterais.

Pausa 2 – após dobrar a face frontal

Pause após o primeiro dobramento. Pergunte: “Onde foi parar o quadrado vermelho? Qual face do cubo ele está formando agora?” A correspondência cromática permite rastrear a resposta.

Pausa 3 – após todas as laterais dobrarem, antes do fechamento

Pause quando o cubo está aberto no topo. Pergunte: “Qual quadrado vai fechar o cubo? De que cor ele é? Onde ele está na planificação agora?”

7.7 Depois de ver

Atividade de consolidação

Distribua folhas com diferentes planificações e peça que os alunos:

1. Identifiquem quais planificações formam um cubo e quais não formam (há planificações com seis quadrados que não dobram corretamente).
2. Para cada planificação válida, marquem quais quadrados ficariam em posições opostas no cubo montado.
3. Recortem e dobrem pelo menos uma das planificações para verificar.

A atividade de recorte e dobramento é insubstituível: ela transfere a visualização da animação para a experiência física, consolidando a compreensão de modo que a representação esquemática e a experiência cinestésica se integrem.

Expansão para outros sólidos

O mesmo princípio de animação se aplica a qualquer sólido com planificação conhecida. Para o tetraedro regular, substitua os **Square** por **Triangle** e ajuste os ângulos de dobramento para 70.5° (o ângulo diedro do tetraedro). Para o cilindro, a planificação envolve um retângulo e dois círculos – a animação do enrolamento do retângulo é particularmente eficaz para mostrar a relação entre o comprimento do retângulo e a circunferência da base.

7.8 Desafios para o professor

1. Crie uma animação da planificação do tetraedro regular. Pesquise o ângulo diedro do tetraedro e use **Triangle** em vez de **Square**.
2. Crie uma versão da animação que mostre o processo inverso: o cubo montado se abrindo até virar planificação. Dica: use **angle=-PI/2** nos mesmos **Rotate** da animação original.
3. Crie uma animação que exiba duas planificações lado a lado – uma válida e uma inválida para formar um cubo – e peça que os alunos identifiquem qual é qual antes de ver o resultado.

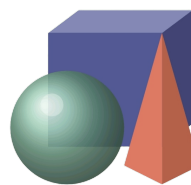


▶ Video 7.1

D2 – Planificação do cubo: dobramento progressivo

https://youtu.be/INSERIR_URL_D2

Ampliação e Redução: D5



A REGRA DE TRÊS é um dos algoritmos mais ensinados e menos compreendidos do ensino fundamental. Os alunos a executam corretamente em exercícios de estrutura familiar, mas não sabem quando aplicá-la em situações novas – nem por que ela funciona. O problema está na dissociação entre o procedimento e o conceito que ele expressa: proporcionalidade descreve como duas grandezas variam em conjunto. Quando uma dobra, a outra dobra. Quando uma triplica, a outra triplica. A razão entre elas permanece constante. A animação torna essa constância diretamente visível: o ponto se move sobre a reta, os valores das duas grandezas crescem simultaneamente, e a razão exibida em tempo real não muda. É essa constância que queríamos mostrar. O aluno não precisa deduzir a constância a partir de um cálculo; pode observá-la como propriedade da relação.

8.1 Por que animar este conteúdo?

O ensino de proporcionalidade centrado na regra de três tende a produzir alunos que calculam sem compreender. A consequência mais visível é a dificuldade em distinguir situações de proporcionalidade direta de situações de proporcionalidade inversa – e de situações em que nenhuma das duas se aplica.

A animação aborda o núcleo do problema: mostra a constante de proporcionalidade como invariante da relação entre duas grandezas que variam juntas. Quando o aluno vê que a razão y/x permanece a mesma enquanto os valores de x e y crescem, ele não precisa mais memorizar que “na proporcionalidade direta, quando x dobra, y dobra” – ele pode

reconstruir esse fato a partir da imagem que a animação construiu.

8.2 Atenção: proporcionalidade direta e inversa em sessões separadas



Dica

8.1

Não apresente proporcionalidade direta e inversa na mesma sessão de animação. Embora a comparação entre as duas relações seja pedagogicamente valiosa, exibir as duas animações em sequência imediata pode fazer o aluno confundir as estruturas ao invés de distingui-las.

Use sessões separadas: trabalhe a proporcionalidade direta em uma aula, consolide com atividades e, só numa aula posterior, introduza a inversa. Compare as duas apenas depois que cada uma estiver consolidada individualmente.

8.3 O que a animação mostra

A animação abre com um par de eixos rotulados com duas grandezas proporcionais – por exemplo, litros de combustível (x) e quilômetros percorridos (y). A reta de proporcionalidade $y = 2x$ é traçada em azul desde a origem.

Um ponto amarelo aparece próximo à origem e começa a se deslocar lentamente sobre a reta. No canto superior direito da tela, a razão y/x é exibida em tempo real, com os valores numéricos atualizando a cada quadro. O aluno observa simultaneamente: o ponto avançando sobre a reta, os valores de x e y crescendo, e a razão y/x permanecendo constante em 2,0.

A animação pausa três vezes em posições distintas da reta para que o aluno leia as coordenadas e confirme a constância da razão. Ao final, a constante $k = 2$ é destacada como a inclinação da reta – conectando a representação geométrica com a expressão algébrica $y = kx$.

8.4 Antes de exibir

O que o aluno precisa saber

Esta animação pressupõe o domínio do plano cartesiano (D9): o aluno precisa saber ler as coordenadas de um ponto e compreender que os eixos representam grandezas. Sem isso, o ponto em movimento será processado como evento visual sem conexão com os valores numéricos exibidos.

Antes de exibir, verifique: o aluno consegue ler as coordenadas de um ponto marcado no plano? Se não, retome o D9 antes de avançar para este capítulo.

Atividade de aquecimento

Proponha oralmente, antes de abrir o computador:

- “Um carro percorre 80 km com 4 litros. Quantos km percorre com 8 litros?”
- “E com 6 litros?”
- “Há uma relação entre a quantidade de litros e os km percorridos? Como você descreveria essa relação?”

Anote as respostas no quadro sem corrigi-las. A animação vai fornecer a linguagem matemática para o que os alunos descreveram intuitivamente.

8.5 O código comentado

O código usa `ValueTracker` e `always_redraw` – os mesmos recursos que introduzimos no D6. Aqui eles servem para mover o ponto sobre a reta e atualizar a razão em tempo real simultaneamente.

```

1  from manim import *
2
3  class D5_Final(Scene):
4      """
5      Conceito: Ampliação e redução de figuras poligonais em malha
6      Nível: Ensino Fundamental - 9º Ano
7      Objetivo: Reconhecer como lados, perímetro e área mudam
8                  ao ampliar ou reduzir uma figura na malha quadriculada.
9      Cenas:
10         1 Apresentação do D5
11         2 Figura original - lados, perímetro e área com cálculos
12         3 Ampliação fator 2 - evidencia perímetro dobra e área quadruplica
13         4 Redução fator  $\frac{1}{2}$  - mesma lógica
14         5 Tabela comparativa + fórmula geral
15         6 Ângulos conservados
16         7 Estratégia para o D5
17     """
18
19     def _limpar(self):
20         self.play(FadeOut(Group(*self.mobjects)), run_time=0.7)
21         self.wait(0.2)
22
23     # -----
24     def _cena1(self):
25         """Apresentação do descritor D5 com texto balanceado."""
26         faixa = Rectangle(width=14.4, height=1.0,
27                             fill_color=DARK_BLUE, fill_opacity=0.9,
28                             stroke_width=0).move_to(UP * 2.8)
29         rotulo = Text("SAEB | Matemática | 9º Ano",
30                         color=COR_ACENTO).scale(0.50)
31         rotulo.move_to(faixa.get_center())
32
33         self.play(FadeIn(cargo), run_time=0.6)
34         self.wait(0.4)
35         simbolo = VGroup(logo_h, logo_v, circ, em)
36         self.play(simbolo.animate.scale(1.06), run_time=0.4)
37         self.play(simbolo.animate.scale(1 / 1.06), run_time=0.35)
38         self.wait(3.5)

```

Código 8.1: D29 – Proporcionalidade direta com razão em tempo real



O método `eixos.coords_to_point(x, y)` converte coordenadas matemáticas em posição na tela – é o equivalente do `plano.c2p()` do `NumberPlane`. Use `coords_to_point` com o objeto `Axes` e `c2p` com o `NumberPlane`: os dois fazem a mesma coisa, mas cada um pertence ao seu tipo de eixo.

8.6 Durante a exibição

Descrever

Pause e pergunte: “Qual é o valor de x ? Qual é o valor de y ?”

Pausa 2 – com o ponto em $x = 3, y = 6$

Pause e pergunte: “ x triplicou em relação à primeira posição. O que aconteceu com y ?”

Pausa 3 – após a constante $k = 2$ aparecer

Pergunte: “O que o número 2 representa geometricamente na reta?” Conecte com o conceito de inclinação: a reta mais inclinada tem k maior.

8.7 Depois de ver

Atividade de consolidação

Distribua uma tabela incompleta de valores de duas grandezas proporcionais e peça que os alunos:

1. Completem a tabela calculando os valores faltantes.
2. Calculem a razão y/x para cada par de valores e verifiquem que é constante.
3. Descrevam em palavras a relação entre as duas grandezas.
4. Escrevam a expressão algébrica $y = kx$.

8.8 Desafios para o professor

1. Adapte a animação da proporcionalidade direta para mostrar duas retas com constantes diferentes ($k = 1$ e $k = 3$) na mesma cena, em cores distintas, e dois pontos se movendo simultaneamente sobre elas.

2. Crie uma animação que mostre uma situação **não** proporcional – por exemplo, $y = 2x + 1$ – e destaque que a razão y/x não é constante. Use isso como contraexemplo para reforçar o conceito.
3. Crie uma animação que conecte a tabela de valores com o gráfico: cada linha da tabela é plotada como um ponto no plano, e os pontos vão se alinhando sobre a reta à medida que a tabela é preenchida.



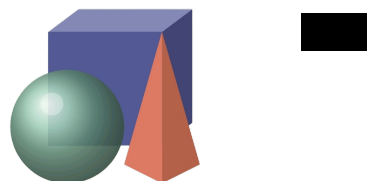
▶ **Video 8.1**

D5-

<https://youtu.be/Qoua3stZG8c>

9

Ângulo como giro: D6



OS ALUNOS aprendem a palavra “ângulo” muito cedo. Aprendem também a desenhá-lo: duas semirretas e um arco entre elas. Mas esse desenho mostra o resultado de um giro, não o giro em si. E é o giro que define o ângulo. Um aluno que só viu ângulos desenhados no papel tem uma concepção figural do conceito – sabe reconhecê-lo numa figura, mas não compreende que ele é uma grandeza que mede o quanto algo girou. A animação corrige isso diretamente: mostra o raio girando e o número crescendo juntos, tornando visível a correspondência entre o processo e a medida.

9.1 Por que animar este conteúdo?

O ângulo, como objeto matemático, é dinâmico por natureza. Representações estáticas mostram o estado depois do giro – o ângulo formado – mas não o processo de girar. Um aluno que só aprende ângulo por imagens fixas recebe o produto de uma transformação sem acesso à transformação em si.

A animação mostra essa transformação diretamente: o raio gira, o arco cresce, e o número que aparece na tela é exatamente o que o transferidor mediu – ou mediria, se o transferidor pudesse acompanhar o movimento. Quando o raio para em 90° , o aluno viu os 90° sendo construídos, não apenas o resultado.

9.2 Recurso central: `ValueTracker` e `always_redraw`

Esta animação introduz dois recursos do Manim que aparecem novamente nos capítulos de D28 (porcentagem) e D29 (proporcionalidade). Vale entendê-los antes de ver o código.

O `ValueTracker` é um objeto especial que guarda um número e nos permite animá-lo suavemente de um valor para outro. Ele não aparece na tela – é invisível. Mas outros objetos podem consultá-lo a cada quadro da animação e se atualizar de acordo com o valor atual.

O `always_redraw` recebe uma função e redesenha o resultado dessa função a cada quadro. Combinado com o `ValueTracker`, ele permite criar objetos que se atualizam em tempo real enquanto o valor rastreado muda.

O par `ValueTracker` + `always_redraw` é a base de qualquer animação em que um número muda e outros elementos precisam acompanhar essa mudança simultaneamente. Usamos esse recurso aqui e nos voltaremos a ele nos capítulos de D28 e D29.

9.3 O que a animação mostra

A animação abre com um ponto central e um raio cinza apontando para a direita, que representa a posição inicial de referência. Um segundo raio azul, inicialmente coincidente com o cinza, começa a girar no sentido anti-horário de forma lenta e contínua. Um arco colorido acompanha o movimento, crescendo à medida que o ângulo aumenta. O valor do ângulo em graus aparece em tempo real próximo à ponta do raio móvel, atualizando-se durante todo o giro.

A animação para brevemente nos ângulos notáveis – 30°, 45°, 60°, 90° e 180° – com uma pausa em cada um para que o aluno associe a posição do raio ao valor numérico. Ao atingir 180°, o raio está na posição oposta ao inicial, completando um ângulo raso.

9.4 Antes de exibir

O que o aluno precisa saber

Esta animação é mais eficaz quando os alunos já conhecem o grau como unidade de medida de ângulo e têm alguma familiaridade com o transferidor – mesmo que básica. Sem essa referência, o número que aparece na tela pode ser processado como informação acessória, sem que o aluno estabeleça a conexão entre o valor e a amplitude do giro.

Se a turma ainda não usou o transferidor, uma sessão prévia de medição de ângulos já desenhados – antes de ver a animação – cria a base necessária.

Atividade de aquecimento

Antes de abrir o computador, use o transferidor no quadro. Desenhe um ângulo de 45° e meça na frente dos alunos, verbalizando cada passo: “Coloco o centro aqui, alinho com o zero, leio onde o outro raio está.” Depois desenhe um ângulo maior, de 120° , e peça que os alunos estimem: “Mais ou menos quantos graus?”

Em seguida, pergunte: “Se o raio continuar girando além de 180° , o ângulo vai ficar maior ou menor?” Essa pergunta prepara a compreensão do ângulo como grandeza contínua e não apenas como figura entre 0° e 180° .

9.5 O código comentado

```
1 from manim import *
2
3 class AnguloComoGiro(Scene):
4     def construct(self):
5
6         centro = ORIGIN
7
8         # Raio fixo de referência: posição inicial (0 graus)
9         raio_ref = Line(centro, RIGHT * 2.5, color=GRAY,
10                        stroke_width=2)
11
12         # Raio móvel: começa na mesma posição do raio fixo
13         raio_mov = Line(centro, RIGHT * 2.5, color=BLUE,
14                        stroke_width=3)
15
16         # ValueTracker: guarda o ângulo atual em graus
17         # Começa em 0 e será animado até o valor desejado
18         angulo_val = ValueTracker(0)
19
20         # Updater do raio móvel: recalcula posição a cada quadro
21         def atualizar_raio(r):
22             ang = angulo_val.get_value()
23             r.put_start_and_end_on(
24                 centro,
25                 centro + 2.5 * np.array([
26                     np.cos(np.radians(ang)),
27                     np.sin(np.radians(ang)),
28                     0
29                 ])
30             )
31
32         raio_mov.add_updater(atualizar_raio)
33
34         # Rótulo numérico: sempre redesenhado com o valor atual
35         # DecimalNumber exibe o número com casas decimais
36         rotulo = always_redraw(lambda: DecimalNumber(
37             angulo_val.get_value(),
38             num_decimal_places=0,
39             color=BLUE
40         ).scale(0.9).next_to(raio_mov.get_end(), UR, buff=0.2))
```



O `always_redraw` recebe uma função lambda (sem parênteses depois de `lambda`). Essa função é chamada a cada quadro, criando um novo objeto com base no valor atual do `ValueTracker`. Se você esquecer o `always_redraw` e criar o arco e o rótulo diretamente, eles ficarão presos no valor inicial e não se atualizarão durante a animação. Esse é o erro mais comum ao usar `ValueTracker`.

9.6 Durante a exibição

Pausa 1 – ao atingir 30°

Pause quando o rótulo 30° aparecer. Pergunte: “Alguém reconhece esse ângulo? Onde já viram um ângulo de 30 graus?” Conecte com experiências concretas: relógio mostrando 1 hora (o ponteiro grande e o pequeno formam aproximadamente 30°), inclinação de rampa, esquadros.

Pausa 2 – ao atingir 90°

Pause quando o raio estiver na posição vertical e o rótulo mostrar 90°. Pergunte: “Esse ângulo tem um nome especial. Alguém sabe qual é?” Se os alunos responderem “ângulo reto”, peça que mostrem exemplos na sala de aula – cantos de mesa, porta, quadro. Essa associação entre o valor numérico, o nome e a percepção visual do ângulo reto é o núcleo do D6.

Pausa 3 – ao atingir 180°

Pause com o raio apontando para a esquerda. Pergunte: “O raio girou metade de uma volta completa. Quantos graus tem uma volta completa?” Isso prepara o conceito de ângulo de 360° sem precisar animá-lo neste momento.

9.7 Depois de ver

Atividade de consolidação

Distribua transferidores e peça que os alunos façam dois tipos de atividade em sequência, para consolidar o procedimento nos dois sentidos:

1. **Medir:** o professor desenha no quadro ou na folha ângulos sem marcação. Os alunos medem com o transferidor e registram os valores.

2. **Construir:** o professor dita valores – “construa um ângulo de 60° ”, “construa um ângulo de 135° ” – e os alunos usam o transferidor para construir.

A animação mostrou a correspondência entre giro e medida; o transferidor físico consolida essa correspondência no sentido prático. A animação não substitui o transferidor – prepara o aluno para usá-lo com compreensão.

Variação: explorar ângulos maiores que 180°

Para turmas que já dominam o ângulo reto e o ângulo raso, adapte o código para mostrar o giro até 270° e 360° , introduzindo ângulos obtuso e ângulo completo:

```
1  # Após a pausa em 180 graus, continue:
2
3  # Giro até 270 graus - ângulo obtuso maior
4  self.play(angulo_val.animate.set_value(270), run_time=3)
5  label_270 = MathTex(" $270^\circ$ ", color=YELLOW).scale(0.7)
6  label_270.next_to(arco, LEFT, buff=0.1)
7  self.play(FadeIn(label_270))
8  self.wait(1)
9  self.play(FadeOut(label_270))
10
11 # Volta completa: 360 graus
12 self.play(angulo_val.animate.set_value(360), run_time=3)
13 label_360 = MathTex(" $360^\circ$ \\text{ = volta completa}",
14                     color=YELLOW).scale(0.7)
15 label_360.to_edge(DOWN)
16 self.play(Write(label_360))
17 self.wait(2)
```

Código 9.2: D6 – Variação com ângulos maiores que 180°

9.8 Desafios para o professor

1. Adapte a animação para mostrar dois raios girando simultaneamente, com velocidades diferentes. Use dois `ValueTracker` separados e observe como os ângulos crescem em ritmos distintos.
2. Crie uma animação que mostre o ponteiro de um relógio girando. O ponteiro dos minutos gira 6° por minuto ($360^\circ \div 60$). Anime o giro de 0 a 360° com `run_time=5` e exiba o número de minutos equivalente ao lado.
3. Crie uma composição de ângulos: mostre dois ângulos consecutivos sendo adicionados, com o arco de cada um numa cor diferente e a soma exibida ao final.



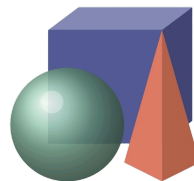
▶ Video 9.1

D6 – Ângulo como giro com medida em tempo real

https://youtu.be/INSEIRU_RLD6

10

Coordenadas no plano cartesiano: D9



PLOTAR um ponto no plano cartesiano parece simples para quem já sabe fazer. Para quem está aprendendo, é um procedimento de duas etapas que precisa ser executado na ordem certa: primeiro vai na horizontal, depois na vertical. Ou é o contrário? E como eu sei onde o número 3 está no eixo x ? E o número -2 ? A animação não torna o procedimento mais fácil – torna o procedimento visível. E quando um procedimento pode ser visto, ele pode ser compreendido. O aluno pode ver as duas projeções sendo traçadas em sequência, cada uma parando no eixo correto, antes de o par ordenado ser montado. Não é necessário memorizar a ordem; é possível entendê-la.

10.1 Por que animar este conteúdo?

Ler as coordenadas de um ponto no plano exige que o aluno realize internamente uma projeção sobre cada eixo – um processo de inferência que não está visível em nenhuma representação estática. O professor pode explicar verbalmente que “você olha para baixo até o eixo x e para a esquerda até o eixo y ”, mas o aluno que ainda não automatizou esse procedimento precisa construí-lo como sequência de operações, não decorá-lo como instrução.

A animação mostra as linhas de projeção sendo traçadas na ordem correta, com cores diferentes para cada eixo, e exibe o valor numérico no momento exato em que a projeção

chega ao eixo. O aluno não precisa imaginar o processo – pode observá-lo.

10.2 O que a animação mostra

A animação abre com o plano cartesiano vazio, com os dois eixos e a grade discreta. Um ponto amarelo aparece na posição $(3, 4)$ e permanece estático por um segundo.

Em seguida, uma linha tracejada vermelha desce verticalmente do ponto até o eixo x , parando sobre o valor 3, que aparece em vermelho abaixo do eixo. Depois de uma breve pausa, uma linha tracejada azul parte horizontalmente do ponto até o eixo y , parando sobre o valor 4, que aparece em azul à esquerda do eixo. O par ordenado $(3, 4)$ aparece então próximo ao ponto, com o 3 em vermelho e o 4 em azul – as mesmas cores das projeções que os geraram.

A cena é reiniciada com um segundo ponto em quadrante diferente, por exemplo $(-2, 3)$, repetindo o procedimento completo. Essa repetição reforça que o processo é o mesmo para qualquer posição no plano.

10.3 Antes de exhibir

O que o aluno precisa saber

Esta animação exige que o aluno já compreenda número negativo e saiba localizar números positivos e negativos em uma reta numérica unidimensional. Sem isso, as projeções sobre os eixos serão observadas como linhas que chegam a posições arbitrárias, sem conexão com os valores numéricos.

O professor deve verificar previamente se a turma opera com confiança sobre a reta numérica. Se houver insegurança, vale trabalhar o eixo x isoladamente – localizar pontos em uma reta horizontal com números negativos – antes de introduzir o plano completo.

Atividade de aquecimento

No papel quadriculado ou no quadro, desenhe o plano cartesiano e marque três pontos sem dar as coordenadas. Peça que os alunos escrevam as coordenadas de cada ponto. Em seguida, dite três pares ordenados e peça que marquem os pontos correspondentes.

Se a turma tiver dificuldade com a atividade oral, este é o sinal de que a animação pode ajudar – mas que precisará de mais mediação durante a exibição. Se a turma resolver com facilidade, a animação serve para formalizar e consolidar o procedimento que eles já praticam intuitivamente.

10.4 O código comentado

```
1 from manim import *
2
3 class CoordenadasCartesianas(Scene):
4     def construct(self):
5
6         # Plano cartesiano com grade de baixa opacidade
7         plano = NumberPlane(
8             x_range=[-5, 5, 1],
9             y_range=[-5, 5, 1],
10            background_line_style={"stroke_opacity": 0.25}
11        )
12        self.add(plano)
13
14        # Ponto a ser identificado: posição (3, 4)
15        P = Dot(plano.c2p(3, 4), color=YELLOW, radius=0.12)
16        self.add(P)
17        self.wait(1)
18
19        # --- Projeção sobre o eixo x ---
20        # Linha vertical tracejada: desce do ponto até o eixo x
21        proj_x = DashedLine(
22            plano.c2p(3, 4),    # ponto de origem
23            plano.c2p(3, 0),    # ponto no eixo x
24            color=RED,
25            dash_length=0.15
26        )
27        # Rótulo: valor 3 aparece abaixo do eixo x
28        label_x = MathTex("3", color=RED).next_to(
29            plano.c2p(3, 0), DOWN, buff=0.2
30        )
31
32        # Primeiro: traçar a projeção (1.5s)
33        # Segundo: revelar o valor (pausa de 0.5s)
34        self.play(Create(proj_x), run_time=1.5)
35        self.play(FadeIn(label_x))
36        self.wait(0.5)    # ponto de pausa (ver abaixo)
37
38        # --- Projeção sobre o eixo y ---
39        # Linha horizontal tracejada: vai do ponto até o eixo y
40        proj_y = DashedLine(
```




O **MathTex** aceita uma lista de strings que são concatenadas. Isso permite colorir partes diferentes do par ordenado com cores diferentes: `par[1].set_color(RED)` acessa o segundo elemento da lista (índice 1), que é o “3”. Essa técnica de indexação funciona para qualquer expressão **MathTex** com múltiplos elementos – útil sempre que você quiser destacar partes específicas de uma fórmula.

10.5 Durante a exibição

Pausa 1 – após a primeira projeção, antes da segunda

Pause quando a linha vermelha chegar ao eixo x e o valor 3 aparecer. Pergunte: “O que essa linha vermelha está fazendo? Onde ela parou? O que o número 3 indica sobre a posição do ponto?”

Esta pausa é o momento mais importante da exibição. O aluno precisa verbalizar o que a projeção significa antes de ver a segunda. Se a resposta for vaga (“a linha foi para baixo”), reformule: “Foi para baixo até onde? E o número que apareceu, o que ele mede?”

Pausa 2 – após o par ordenado aparecer

Pause com o par $(3, 4)$ visível, com as cores. Pergunte: “Por que o 3 está em vermelho e o 4 em azul?” e “Se o ponto estivesse em $(-2, 3)$, como seria o processo?” Deixe os alunos responderem antes de exibir a segunda parte da animação.

10.6 Depois de ver

Atividade de consolidação

A animação trabalhou o procedimento em um sentido: do ponto para as coordenadas. A consolidação deve trabalhar os dois sentidos:

1. **Do ponto ao par ordenado:** no papel quadriculado, o professor marca três pontos e os alunos escrevem as coordenadas, traçando as linhas de projeção com régua – reproduzindo fisicamente o que a animação mostrou.
2. **Do par ordenado ao ponto:** o professor dita pares ordenados e os alunos localizam os pontos, usando o mesmo procedimento em sentido inverso.

O movimento físico de traçar a projeção com régua é insubstituível: ele reforça a sequência procedimental que a animação tornou visível.

Variação: o processo inverso

A animação principal mostra como ir do ponto ao par ordenado. Esta variação mostra o processo inverso: dado o par $(5, 3)$, como encontrar o ponto?

```

1  from manim import *
2
3  class CoordenadaParaPonto(Scene):
4      def construct(self):
5
6          plano = NumberPlane(
7              x_range=[-5, 5, 1],
8              y_range=[-5, 5, 1],
9              background_line_style={"stroke_opacity": 0.25}
10         )
11         self.add(plano)
12
13         # Exibe o par ordenado como ponto de partida
14         par = MathTex(r"(\,, \"5\", r\"\",,\", \"3\", r\"\",)\")
15         par[1].set_color(RED)
16         par[3].set_color(BLUE)
17         par.to_corner(UL)
18         self.play(Write(par))
19         self.wait(1)
20
21         # Marca o 5 no eixo x (partida da projeção)
22         marca_x = Dot(plano.c2p(5, 0), color=RED, radius=0.1)
23         self.play(GrowFromCenter(marca_x))
24
25         # Linha vertical sobe do eixo x até a altura y=3
26         linha_x = DashedLine(
27             plano.c2p(5, 0),
28             plano.c2p(5, 3),
29             color=RED, dash_length=0.15
30         )
31         self.play(Create(linha_x), run_time=1.5)
32
33         # Marca o 3 no eixo y
34         marca_y = Dot(plano.c2p(0, 3), color=BLUE, radius=0.1)
35         self.play(GrowFromCenter(marca_y))
36
37         # Linha horizontal vai do eixo y até x=5
38         linha_y = DashedLine(
39             plano.c2p(0, 3),
40             plano.c2p(5, 3),
41             color=BLUE, dash_length=0.15
42         )
43         self.play(Create(linha_y), run_time=1.5)

```

Exibir as duas versões na mesma aula – primeiro do ponto ao par, depois do par ao ponto – reforça que o par ordenado e o ponto são duas formas de representar a mesma informação. Essa bidirecionalidade é o núcleo do D9.

10.7 Desafios para o professor

1. Crie uma animação que mostre quatro pontos em quadrantes diferentes sendo identificados em sequência, usando a mesma lógica de projeção. Observe como os sinais das coordenadas variam conforme o quadrante.
2. Adapte a animação para mostrar pontos sobre os eixos – como $(3, 0)$ e $(0, -2)$ – e mostre que um dos eixos é “transparente” para a projeção nesses casos.
3. Crie uma versão interativa em que o ponto se move e as projeções acompanham o movimento usando `always_redraw` e `ValueTracker` (como no D6). As coordenadas devem se atualizar em tempo real enquanto o ponto se desloca.



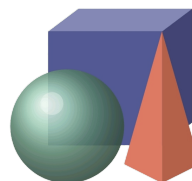
▶ Video 10.1

D9 – Coordenadas cartesianas por projeção sequencial

<https://youtu.be/INSEIR_URL_D9>

11

Números inteiros na reta numérica: D16



NÚMEROS NEGATIVOS são um dos obstáculos mais consistentes no percurso matemático dos anos finais do ensino fundamental. Não porque o conceito seja intrinsecamente difícil, mas porque ele desafia a intuição construída durante anos de trabalho apenas com naturais: números que crescem para a direita, em que “maior” sempre significa “mais à frente”. Os negativos invertem essa intuição. A reta numérica é o instrumento que permite reconstruir a intuição correta, porque torna visível a simetria em relação ao zero. Mas uma reta estática mostra onde os números estão – não o que acontece quando você opera com eles. A animação mostra o que acontece: o ponto parte, percorre a reta, cruza o zero, entra no território negativo. O aluno não precisa imaginar; pode ver.

11.1 Por que animar este conteúdo?

A dificuldade com inteiros negativos raramente é de definição – é de imagem espacial. O aluno sabe que -3 é “menos três”, mas não tem uma imagem clara de onde -3 está em relação a $+3$ e a 0 , e de como essas posições se relacionam com as operações de adição e subtração.

A reta estática com números marcados mostra as posições, mas não as relações de movimento que dão sentido às operações. Quando o professor diz “somar um número negativo é se deslocar para a esquerda”, o aluno precisa imaginar esse deslocamento. A

animação dispensa essa inferência: o deslocamento acontece na tela, incluindo o momento em que o ponto cruza o zero – que é a fronteira mais importante do conceito.

11.2 Em uma sessão ou em duas?

Esta é a única animação deste e-book com uma recomendação específica de ritmo de uso que varia conforme o perfil da turma. Vale ler com atenção antes de decidir como usar.



Dica

11.1

Se sua turma já tem familiaridade com números negativos – mesmo que básica – exiba a animação completa de uma vez: os dois deslocamentos, incluindo a passagem pelo zero.

Se a turma está sendo apresentada ao conceito de número negativo pela **primeira vez**, considere dividir em duas sessões. Na primeira, exiba apenas o primeiro deslocamento (positivo, dentro do território dos naturais) e trabalhe a reta como ferramenta de raciocínio. Na segunda sessão, retome e exiba o deslocamento que cruza o zero. A fronteira entre positivos e negativos merece atenção separada, não pode ser passada rapidamente junto com o resto.

11.3 O que a animação mostra

A animação abre com uma reta numérica de -6 a 6 . O zero está marcado em branco para se destacar como fronteira. Um ponto amarelo aparece sobre o zero.

O texto “ $+3$ ” aparece acima da reta e o ponto se desloca três unidades para a direita, percorrendo o trecho positivo da reta. Uma pausa de meio segundo marca a chegada ao $+3$.

Em seguida, o texto “ -5 ” aparece e o ponto se desloca cinco unidades para a esquerda – mais devagar que o primeiro deslocamento. A animação destaca explicitamente o momento em que o ponto cruza o zero: um círculo branco pulsa brevemente sobre o zero e a velocidade reduz ainda mais enquanto o ponto passa pela fronteira. O ponto para sobre o -2 .

A operação completa $3 + (-5) = -2$ é exibida abaixo da reta, conectando o que foi visto visualmente com a notação algébrica.

11.4 Antes de exhibir

O que o aluno precisa saber

Esta animação pressupõe que o aluno já opera com fluência sobre a reta dos naturais: sabe que os números crescem para a direita, que a distância entre números consecutivos é sempre 1 e que a posição na reta corresponde ao valor do número.

Se o aluno ainda não internalizou a reta numérica como representação espacial de grandezas, ele vai focar no movimento do ponto como evento visual sem estabelecer a conexão com a operação aritmética. Antes de usar esta animação, vale verificar: o aluno consegue dizer, sem contar um a um, qual número fica 4 posições à direita do 5? Se não, a reta dos naturais precisa de mais trabalho antes de introduzir os negativos.

Atividade de aquecimento

Desenhe a reta numérica no quadro, de -6 a 6 , sem marcar o ponto. Faça três perguntas orais antes de abrir o computador:

- “Se eu partir do zero e andar 3 passos para a direita, onde chego?”
- “Agora, do 3, se eu andar 5 passos para a esquerda, onde chego?”
- “Alguém consegue escrever essa operação com números?”

Deixe os alunos responderem e discuta as respostas antes de exhibir a animação. A confrontação entre a previsão oral e o que a animação mostra é mais eficaz do que a observação passiva.

11.5 O código comentado

```
1 from manim import *
2
3 class InteirosNaReta(Scene):
4     def construct(self):
5
6         # Reta numérica de -6 a 6
7         # include_numbers exibe os valores em cada marca
8         reta = NumberLine(
9             x_range=[-6, 6, 1],
10            length=12,
11            include_numbers=True,
12            color=GRAY
13        )
14        self.add(reta)
15
16        # Destaque do zero: marca a fronteira positivo/negativo
17        marca_zero = Dot(reta.n2p(0), color=WHITE, radius=0.12)
18        self.add(marca_zero)
19
20        # Ponto inicial no zero
21        ponto = Dot(reta.n2p(0), color=YELLOW, radius=0.15)
22        self.add(ponto)
23        self.wait(1)
24
25        # --- Primeiro deslocamento: +3, para a direita ---
26        texto1 = MathTex("+3", color=GREEN).scale(1.2)
27        texto1.next_to(reta, UP, buff=0.5)
28        self.play(Write(texto1))
29
30        # run_time=2: ~0.7 seg por unidade, permite contar os passos
31        self.play(
32            ponto.animate.move_to(reta.n2p(3)),
33            run_time=2
34        )
35        self.wait(0.5)
36
37        # --- Segundo deslocamento: -5, cruza o zero ---
38        texto2 = MathTex("-5", color=RED).scale(1.2)
39        texto2.next_to(reta, UP, buff=0.5)
40        self.play(Transform(texto1, texto2))
```



O `run_time=3` no segundo deslocamento é uma decisão pedagógica, não técnica. O primeiro deslocamento é positivo e intuitivo – o aluno já sabe que somar move para a direita. O segundo cruza a fronteira do zero, que é o momento mais exigente cognitivamente. Dar mais tempo a esse deslocamento é o código dizendo ao aluno: “preste atenção, algo importante vai acontecer aqui”.

11.6 Durante a exibição

Pausa 1 – após o ponto parar em +3

Pause com o ponto sobre o +3. Pergunte: “Quantas unidades o ponto andou? Em que direção? Qual é o sinal da operação que corresponde a andar para a direita?” Conecte explicitamente: sinal positivo, direção direita, número maior que zero.

Pausa 2 – com o ponto sobre o zero, durante o segundo deslocamento

Esta pausa requer atenção: o ponto passa pelo zero sem parar. Se você quiser pausar exatamente nesse momento, assista à animação antes da aula para saber em que segundo o ponto cruza o zero. Pause e pergunte: “O ponto está no zero. Ainda vai para a esquerda. Para onde ele vai agora? Que tipo de número fica à esquerda do zero?”

Pausa 3 – após o resultado $3 + (-5) = -2$ aparecer

Pergunte: “O resultado é negativo. Isso faz sentido olhando para a posição do ponto na reta?” Peça que os alunos formulem uma regra a partir do que viram: quando somamos um número negativo com módulo maior que o positivo, o resultado é negativo porque...

11.7 Depois de ver

Atividade de consolidação

Distribua fichas com retas numéricas de -8 a 8 impressas. Proponha três operações em sequência, pedindo que os alunos façam duas coisas antes de calcular:

1. **Prever:** escrever se o resultado será positivo, negativo ou zero, sem calcular.
2. **Traçar:** desenhar o deslocamento na reta com setas, indicando de onde parte e para onde vai.
3. **Calcular:** registrar o resultado numérico.

Sugestão de operações: $5 + (-8)$, $-3 + 7$, $-4 + (-2)$.

A sequência prever traçar calcular transforma a reta de referência visual em ferramenta de raciocínio, que é o objetivo do D16.

Versão abreviada para primeira sessão

Para turmas sem contato anterior com negativos, use esta versão que mostra apenas o primeiro deslocamento e para antes de cruzar o zero:

```

1  from manim import *
2
3  class InteirosNaRetaSessao1(Scene):
4      def construct(self):
5
6          reta = NumberLine(
7              x_range=[-6, 6, 1],
8              length=12,
9              include_numbers=True,
10             color=GRAY
11         )
12         self.add(reta)
13
14         ponto = Dot(reta.n2p(0), color=YELLOW, radius=0.15)
15         self.add(ponto)
16         self.wait(1)
17
18         # Apenas o deslocamento positivo na primeira sessão
19         texto = MathTex("+4", color=GREEN).scale(1.2)
20         texto.next_to(reta, UP, buff=0.5)
21         self.play(Write(texto))
22         self.play(
23             ponto.animate.move_to(reta.n2p(4)),
24             run_time=2.5
25         )
26         self.wait(1)
27
28         resultado = MathTex("0 + 4 = 4").scale(1.1)
29         resultado.next_to(reta, DOWN, buff=0.5)
30         self.play(Write(resultado))
31         self.wait(2)

```

Código 11.2: D16 – Versão para primeira sessão (somente positivos)

11.8 Desafios para o professor

1. Crie uma animação que mostre subtração de inteiros como adição do oposto: $5 - 8 = 5 + (-8)$. Mostre os dois deslocamentos na mesma reta, com cores distintas para o positivo e o negativo.
2. Adapte a animação para mostrar três operações consecutivas em sequência, acumulando deslocamentos. Mantenha o rastro de cada deslocamento com uma cor diferente usando `TracedPath` (como no D1).
3. Crie uma versão que mostre o valor absoluto visualmente: dois pontos simétricos em relação ao zero, com uma seta indicando que ambos estão à mesma distância do zero.



▶ Video 11.1

D16 – Inteiros na reta: deslocamento e passagem pelo zero

https://youtu.be/INSERIR_U RL_D16

12

Porcentagem como relação parte-todo: D28



A LUNO que sabe calcular 25% de 80 nem sempre sabe o que 25% significa. O procedimento – dividir por 100, multiplicar pela parte – funciona mecanicamente. Mas quando a situação muda um pouco: “25% de 200 é quanto?”, ou “de quanto por cento 50 representa em relação a 200?”, a segurança some. Isso acontece porque o algoritmo foi aprendido sem a compreensão da relação que ele expressa: porcentagem é uma razão entre uma parte e um todo, referida a uma escala de cem. Optamos por colocar essa invariância no centro da animação. Quando o todo muda, a razão permanece. A animação torna essa invariância diretamente visível – e é exatamente essa invariância que o algoritmo esconde.

12.1 Por que animar este conteúdo?

O ensino de porcentagem centrado em procedimentos algorítmicos produz alunos que calculam sem compreender. A consequência mais visível é a dificuldade em transferir o procedimento para situações novas: se o problema não tem a mesma estrutura dos exercícios memorizados, o aluno não sabe o que fazer.

A animação aborda o núcleo do problema: mostra que a porcentagem é uma razão invariante sob o reescalonamento do todo. Quando o todo dobra, a parte também dobra – e a porcentagem fica igual. Essa invariância é o que justifica o algoritmo, e é o que o aluno precisa compreender antes de automatizá-lo.

12.2 O que a animação mostra

A animação abre com uma barra horizontal cinza representando o todo (100 unidades). Uma porção azul à esquerda representa 25 unidades – os 25%. Abaixo da barra, o texto “25 em 100 = 25%” aparece.

A barra não muda de tamanho, mas o rótulo abaixo se transforma: agora a barra representa 200 unidades e a porção azul representa 50 – a mesma proporção. O texto mostra “50 em 200 = 25%”. A porcentagem permanece a mesma porque a razão entre parte e todo não mudou.

Uma terceira situação com total de 80 unidades completa a sequência. Aqui, 25% de 80 é 20 – um resultado não inteiro que aparece como razão simplificada. A barra agora está menor, e a porção azul representa 20 unidades.

Ao final, as três barras são exibidas simultaneamente com suas proporções, mostrando visualmente que a mesma fração do todo foi destacada nos três casos.

12.3 Antes de exhibir

O que o aluno precisa saber

Esta animação pressupõe que o aluno compreende fração como relação parte-todo e que reconhece razão como comparação entre duas quantidades. Um aluno que ainda associa fração apenas a “dividir uma coisa em pedaços” pode ter dificuldade em conectar a barra da animação com a ideia de razão.

Antes da exibição, verifique com a turma: conseguem expressar uma fração como razão? Por exemplo: “numa turma de 30 alunos, 18 são meninas. Qual fração da turma é menina?” Se a turma reformula isso como “18 para 30” ou “18 de cada 30”, a base está pronta.

Atividade de aquecimento

Leve para a aula um produto com porcentagem visível na embalagem – um suco “com 30% menos açúcar”, um produto “com 50% de desconto”, qualquer embalagem que use porcentagem em contexto real. Mostre para a turma e pergunte:

- “30% menos açúcar do quê? Em relação a qual quantidade?”
- “Se o produto original tinha 100g de açúcar, quanto tem agora?”
- “E se tivesse 200g? E se tivesse 50g?”

Essa sequência de perguntas antecipa exatamente o que a animação vai mostrar: a porcentagem é uma razão que se mantém constante quando o todo muda proporcionalmente.

12.4 O código comentado

O código da animação principal usa `Rectangle` para as barras e `Transform` para atualizar os rótulos. A porção azul é posicionada com `align_to` para sempre começar na extremidade esquerda da barra total.

```

1  from manim import *
2
3  class Porcentagem(Scene):
4      def construct(self):
5
6          # --- Situação 1: 25 em 100 ---
7
8          # Barra total: representa o "todo"
9          # Largura 8 unidades = 100% da representação
10         barra_total = Rectangle(
11             width=8, height=0.9,
12             color=GRAY, fill_opacity=0.3
13         ).shift(UP * 0.5)
14
15         # Barra parcial: 25% de 8 = 2 unidades de largura
16         barra_parte = Rectangle(
17             width=2, height=0.9,
18             color=BLUE, fill_opacity=0.7
19         ).align_to(barra_total, LEFT).shift(UP * 0.5)
20
21         rotulo1 = MathTex(
22             r"25 \text{ em } 100 = 25\%"
23         ).next_to(barra_total, DOWN, buff=0.3)
24
25         self.play(
26             FadeIn(barra_total),
27             FadeIn(barra_parte),
28             Write(rotulo1)
29         )
30         self.wait(2)    # ponto de pausa (ver abaixo)
31
32         # --- Situação 2: 50 em 200 ---
33         # A barra representa agora 200 unidades
34         # 25% de 200 = 50, mas a proporção visual é a mesma
35         # Porque a largura da parte/total = 2/8 = 25%
36
37         rotulo2 = MathTex(
38             r"50 \text{ em } 200 = 25\%"
39         ).next_to(barra_total, DOWN, buff=0.3)
40
41         # O Transform no rótulo mostra a mudança de escala
42         # mantendo a mesma porcentagem
43         self.play(

```



O `align_to(barra_total, LEFT)` garante que a barra parcial sempre comece na extremidade esquerda da barra total, independentemente das larguras. Sem esse alinhamento, a barra parcial ficaria centralizada dentro da total, o que distorceria a representação visual da razão parte-todo.

12.5 Variação: porcentagem como setor circular

A representação em barra é versátil, mas os alunos reconhecem mais facilmente a porcentagem no “gráfico de pizza” que aparece em jornais e livros. Usar as duas representações na mesma aula reforça que a porcentagem descreve uma relação, não depende do formato visual.

```

1  from manim import *
2
3  class PorcentagemCirculo(Scene):
4      def construct(self):
5
6          # Círculo completo = 100%
7          circulo = Circle(radius=2, color=GRAY,
8                           fill_opacity=0.2)
9
10         # Setor de 25%: 25% de 360° = 90°
11         setor = Sector(
12             radius=2,
13             angle=PI / 2,          # 90 graus = 25%
14             start_angle=0,
15             color=BLUE,
16             fill_opacity=0.7
17         )
18
19         rotulo = MathTex("25\\%", color=BLUE).scale(1.2)
20         rotulo.move_to(setor.get_center() + RIGHT * 0.5)
21
22         self.play(FadeIn(circulo))
23         self.play(Create(setor), Write(rotulo))
24         self.wait(2)
25
26         # Mostrar equivalência com a barra
27         barra_total = Rectangle(width=6, height=0.8,
28                                color=GRAY, fill_opacity=0.2
29                                ).shift(DOWN * 2.5)
30         barra_parte = Rectangle(width=1.5, height=0.8,
31                                color=BLUE, fill_opacity=0.7
32                                ).align_to(barra_total, LEFT
33                                ).shift(DOWN * 2.5)
34         lbl_barra = MathTex(
35             r"25\\% = \frac{1}{4}"
36         ).scale(0.9).next_to(barra_total, RIGHT, buff=0.3)
37
38         self.play(FadeIn(barra_total), FadeIn(barra_parte),
39                   Write(lbl_barra))
40         self.wait(2)

```

12.6 Durante a exibição

Pausa 1 – após a primeira situação (25 em 100)

Pause com a barra e o rótulo visíveis. Pergunte: “O que a parte azul representa? Quanto do total ela ocupa?” Conecte o visual com a linguagem: “25 de cada 100” é o que 25% significa.

Pausa 2 – após a segunda situação (50 em 200)

Pause e pergunte: “A barra azul ficou maior ou menor? E a porcentagem, ficou maior, menor ou igual?” A resposta esperada – a barra azul cresceu, mas a porcentagem ficou igual – é o núcleo do conceito. Se os alunos acharem que a porcentagem deveria ter mudado, isso revela a concepção que a animação pretende corrigir.

Pausa 3 – na composição final com três barras

Pause com as três barras visíveis. Pergunte: “O que há de igual nas três situações? O que é diferente?” A resposta: a proporção visual é igual (a parte ocupa sempre o mesmo quarto da barra), os valores absolutos são diferentes.

12.7 Depois de ver

Atividade de consolidação

Distribua folhas com três barras em branco impressas (como régua sem números). Peça que os alunos marquem 30% em cada barra, sendo que:

- A primeira barra representa um total de 100.
- A segunda representa um total de 50.
- A terceira representa um total de 300.

O aluno que compreendeu a animação vai marcar sempre o mesmo ponto na barra (30% do comprimento total), independentemente dos valores. O aluno que ainda confunde porcentagem com valor absoluto vai marcar pontos diferentes.

Essa atividade torna visível a diferença de compreensão sem precisar de uma prova.

12.8 Desafios para o professor

1. Crie uma animação que mostre 30%, 50% e 75% em três barras simultâneas, para comparação visual das proporções.

2. Adapte a animação do setor circular para usar **ValueTracker**: o setor cresce de 0% a 100% enquanto o rótulo numérico se atualiza em tempo real (como no D6).
3. Crie uma animação que mostre a relação entre fração, decimal e porcentagem: $\frac{1}{4} = 0,25 = 25\%$, com as três representações aparecendo em sequência sobre a mesma barra.



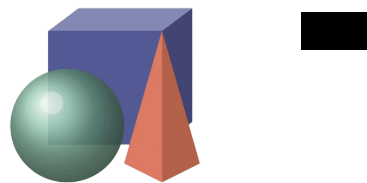
▶ **Video 12.1**

D28 – Porcentagem: invariância da razão parte-todo

https://youtu.be/INSERIR_URL_D28

13

Proporcionalidade: D29



A REGRA DE TRÊS é um dos algoritmos mais ensinados e menos compreendidos do ensino fundamental. Os alunos a executam corretamente em exercícios de estrutura familiar, mas não sabem quando aplicá-la em situações novas – nem por que ela funciona. O problema está na dissociação entre o procedimento e o conceito que ele expressa: proporcionalidade descreve como duas grandezas variam em conjunto. Quando uma dobra, a outra dobra. Quando uma triplica, a outra triplica. A razão entre elas permanece constante. A animação torna essa constância diretamente visível: o ponto se move sobre a reta, os valores das duas grandezas crescem simultaneamente, e a razão exibida em tempo real não muda. É essa constância que queríamos mostrar. O aluno não precisa deduzir a constância a partir de um cálculo; pode observá-la como propriedade da relação.

13.1 Por que animar este conteúdo?

O ensino de proporcionalidade centrado na regra de três tende a produzir alunos que calculam sem compreender. A consequência mais visível é a dificuldade em distinguir situações de proporcionalidade direta de situações de proporcionalidade inversa – e de situações em que nenhuma das duas se aplica.

A animação aborda o núcleo do problema: mostra a constante de proporcionalidade como invariante da relação entre duas grandezas que variam juntas. Quando o aluno vê que a razão y/x permanece a mesma enquanto os valores de x e y crescem, ele não precisa mais memorizar que “na proporcionalidade direta, quando x dobra, y dobra” – ele pode

reconstruir esse fato a partir da imagem que a animação construiu.

13.2 Atenção: proporcionalidade direta e inversa em sessões separadas



Dica

13.1

Não apresente proporcionalidade direta e inversa na mesma sessão de animação. Embora a comparação entre as duas relações seja pedagogicamente valiosa, exibir as duas animações em sequência imediata pode fazer o aluno confundir as estruturas ao invés de distingui-las.

Use sessões separadas: trabalhe a proporcionalidade direta em uma aula, consolide com atividades e, só numa aula posterior, introduza a inversa. Compare as duas apenas depois que cada uma estiver consolidada individualmente.

13.3 O que a animação mostra

A animação abre com um par de eixos rotulados com duas grandezas proporcionais – por exemplo, litros de combustível (x) e quilômetros percorridos (y). A reta de proporcionalidade $y = 2x$ é traçada em azul desde a origem.

Um ponto amarelo aparece próximo à origem e começa a se deslocar lentamente sobre a reta. No canto superior direito da tela, a razão y/x é exibida em tempo real, com os valores numéricos atualizando a cada quadro. O aluno observa simultaneamente: o ponto avançando sobre a reta, os valores de x e y crescendo, e a razão y/x permanecendo constante em 2,0.

A animação pausa três vezes em posições distintas da reta para que o aluno leia as coordenadas e confirme a constância da razão. Ao final, a constante $k = 2$ é destacada como a inclinação da reta – conectando a representação geométrica com a expressão algébrica $y = kx$.

13.4 Antes de exibir

O que o aluno precisa saber

Esta animação pressupõe o domínio do plano cartesiano (D9): o aluno precisa saber ler as coordenadas de um ponto e compreender que os eixos representam grandezas. Sem isso, o ponto em movimento será processado como evento visual sem conexão com os valores numéricos exibidos.

Antes de exibir, verifique: o aluno consegue ler as coordenadas de um ponto marcado no plano? Se não, retome o D9 antes de avançar para este capítulo.

Atividade de aquecimento

Proponha oralmente, antes de abrir o computador:

- “Um carro percorre 80 km com 4 litros. Quantos km percorre com 8 litros?”
- “E com 6 litros?”
- “Há uma relação entre a quantidade de litros e os km percorridos? Como você descreveria essa relação?”

Anote as respostas no quadro sem corrigi-las. A animação vai fornecer a linguagem matemática para o que os alunos descreveram intuitivamente.

13.5 O código comentado

O código usa `ValueTracker` e `always_redraw` – os mesmos recursos que introduzimos no D6. Aqui eles servem para mover o ponto sobre a reta e atualizar a razão em tempo real simultaneamente.

```

1  from manim import *
2
3  class ProporcionalidadeDireta(Scene):
4      def construct(self):
5
6          # Eixos com rótulos de grandezas
7          eixos = Axes(
8              x_range=[0, 6, 1],
9              y_range=[0, 12, 2],
10             x_length=6,
11             y_length=5,
12             axis_config={"include_numbers": True}
13         )
14         rot_x = Text("litros", font_size=20).next_to(
15             eixos.x_axis.get_end(), RIGHT, buff=0.2)
16         rot_y = Text("km", font_size=20).next_to(
17             eixos.y_axis.get_end(), UP, buff=0.2)
18         self.play(Create(eixos), Write(rot_x), Write(rot_y))
19
20         # Reta de proporcionalidade: y = 2x (k = 2)
21         reta = eixos.plot(
22             lambda x: 2 * x,
23             x_range=[0, 5.5],
24             color=BLUE
25         )
26         self.play(Create(reta))
27
28         # ValueTracker: controla a posição x do ponto
29         t = ValueTracker(0.3)
30
31         # Ponto: sempre redesenhado sobre a reta
32         ponto = always_redraw(lambda: Dot(
33             eixos.coords_to_point(
34                 t.get_value(),
35                 2 * t.get_value()
36             ),
37             color=YELLOW, radius=0.1
38         ))
39
40         # Razão y/x em tempo real - atualiza a cada quadro
41         razao = always_redraw(lambda: MathTex(
42             r"\frac{y}{x} = \frac{"
43             + f"{2 * t.get_value():.1f}"

```



O método `eixos.coords_to_point(x, y)` converte coordenadas matemáticas em posição na tela – é o equivalente do `plano.c2p()` do `NumberPlane`. Use `coords_to_point` com o objeto `Axes` e `c2p` com o `NumberPlane`: os dois fazem a mesma coisa, mas cada um pertence ao seu tipo de eixo.

13.6 Proporcionalidade inversa: código separado

A proporcionalidade inversa tem estrutura diferente: quando x dobra, y cai pela metade. O gráfico não é uma reta – é uma hipérbole. Use esta animação em sessão posterior, após consolidar a direta.

```

1  from manim import *
2
3  class ProporcionalidadeInversa(Scene):
4      def construct(self):
5
6          eixos = Axes(
7              x_range=[0, 7, 1],
8              y_range=[0, 7, 1],
9              x_length=6,
10             y_length=5,
11             axis_config={"include_numbers": True}
12         )
13         self.play(Create(eixos))
14
15         # Hipérbole:  $y = 6/x$  (produto  $xy = 6 = \text{constante}$ )
16         curva = eixos.plot(
17             lambda x: 6 / x if x > 0.3 else 20,
18             x_range=[0.4, 6],
19             color=RED
20         )
21         self.play(Create(curva))
22
23         t = ValueTracker(1)
24
25         ponto = always_redraw(lambda: Dot(
26             eixos.coords_to_point(
27                 t.get_value(),
28                 6 / t.get_value()
29             ),
30             color=YELLOW, radius=0.1
31         ))
32
33         # Produto  $xy$ : permanece constante = 6
34         produto = always_redraw(lambda: MathTex(
35             r"x \cdot y = "
36             + f"{t.get_value():.1f}"
37             + r" \times "
38             + f"{6 / t.get_value():.1f}"
39             + r" = 6{,}0",
40             color=YELLOW
41         ).to_corner(UR).scale(0.65))
42
43         self.add(ponto, produto)

```

Na proporcionalidade direta, a **razão** y/x é constante e o gráfico é uma reta. Na inversa, o **produto** $x \cdot y$ é constante e o gráfico é uma hipérbole. Essa diferença estrutural, que o código torna visível, é o núcleo da distinção entre as duas relações.

13.7 Durante a exibição

Pausa 1 – com o ponto em $x = 1, y = 2$

Pause e pergunte: “Qual é o valor de x ? Qual é o valor de y ? Qual é a razão y/x ?” Peça que os alunos calculem manualmente antes de olhar para o canto da tela. Compare a resposta deles com o valor exibido.

Pausa 2 – com o ponto em $x = 3, y = 6$

Pause e pergunte: “ x triplicou em relação à primeira posição. O que aconteceu com y ? E com a razão?” Conecte explicitamente: quando x multiplica por 3, y também multiplica por 3 – a razão não muda.

Pausa 3 – após a constante $k = 2$ aparecer

Pergunte: “O que o número 2 representa geometricamente na reta?” Conecte com o conceito de inclinação: a reta mais inclinada tem k maior.

13.8 Depois de ver

Atividade de consolidação

Distribua uma tabela incompleta de valores de duas grandezas proporcionais e peça que os alunos:

1. Completem a tabela calculando os valores faltantes.
2. Calculem a razão y/x para cada par de valores e verifiquem que é constante.
3. Descrevam em palavras a relação entre as duas grandezas.
4. Escrevam a expressão algébrica $y = kx$.

A sequência tabela razão palavras expressão reproduz o percurso que a animação iniciou visualmente, consolidando-o nos registros numérico, verbal e algébrico.

13.9 Desafios para o professor

1. Adapte a animação da proporcionalidade direta para mostrar duas retas com constantes diferentes ($k = 1$ e $k = 3$) na mesma cena, em cores distintas, e dois pontos se movendo simultaneamente sobre elas.

2. Crie uma animação que mostre uma situação **não** proporcional – por exemplo, $y = 2x + 1$ – e destaque que a razão y/x não é constante. Use isso como contraexemplo para reforçar o conceito.
3. Crie uma animação que conecte a tabela de valores com o gráfico: cada linha da tabela é plotada como um ponto no plano, e os pontos vão se alinhando sobre a reta à medida que a tabela é preenchida.



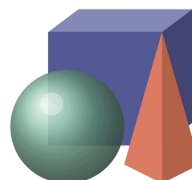
▶ **Video 13.1**

D29 – Proporcionalidade direta: razão constante

https://youtu.be/INSEIR_URL_D29

14

Descobrendo padrões: D32



ANTES de a álgebra fazer sentido, é preciso perceber que há padrões para nomear. Um aluno que olha para 2, 4, 6, 8, ... e reconhece que sempre aumenta de 2 está no caminho certo – mas ainda não chegou ao ponto em que pode responder: “qual é o 100 termo?” sem calcular todos os anteriores. É esse salto, da percepção do padrão para a expressão algébrica da regra geral, que o D32 exige. E é exatamente esse processo que a animação torna observável: o aluno não apenas vê os termos – vê cada termo sendo construído a partir do anterior, com o mecanismo de crescimento destacado em cor contrastante. Quando o mecanismo é visível, a regra geral pode ser formulada.

14.1 Por que animar este conteúdo?

Representações estáticas de sequências mostram os primeiros termos como lista pronta. O aluno precisa inferir a regra de formação a partir de casos particulares – um processo de indução que muitos não conseguem fazer de forma autônoma.

A animação inverte esse processo: em vez de mostrar os termos e pedir a regra, ela constrói cada termo diante do aluno, destacando em vermelho exatamente o que foi acrescentado. O mecanismo de crescimento deixa de ser implícito e passa a ser objeto direto de observação. Quando a regra geral aparece ao final, o aluno a reconhece como descrição do que acabou de ver.

14.2 Antes de exhibir

O que o aluno precisa saber

Esta animação pressupõe que o aluno opera com fluência em operações com números naturais e tem noção informal de sequência como lista ordenada com alguma regularidade. Não é necessário conhecimento prévio de álgebra – o objetivo da animação é justamente construir a ponte para a linguagem algébrica.

Antes de exhibir, verifique: o aluno consegue identificar o próximo termo de uma sequência simples e explicar por quê? Se a resposta for “sim, é só contar”, sem conseguir articular a regra, a animação vai ajudar a formalizar o que ele faz intuitivamente.

Atividade de aquecimento

Escreva no quadro a sequência $2, 5, 8, 11, \dots$ e pergunte:

- “Qual é o próximo número? E o seguinte?”
- “Quanto aumenta de um termo para o próximo?”
- “Como você saberia qual é o 100 termo sem calcular todos os anteriores?”

A última pergunta é a mais importante – e a que a maioria dos alunos não consegue responder ainda. Anote as tentativas no quadro sem corrigi-las. A animação vai fornecer o caminho para esse problema.

A animação não deve ser exibida antes que os alunos tenham tentado por conta própria responder à terceira pergunta. A generalização se torna muito mais significativa quando o aluno já sentiu a dificuldade de chegar lá sem a regra.

14.3 Sequências numéricas

Nos exemplos abaixo, os termos aparecem um a um com seus índices. Ao final, a expressão algébrica surge em destaque – como resposta à pergunta que ficou em aberto na atividade de aquecimento.



[Sequência aritmética] Uma sequência aritmética é uma lista ordenada de números em que a diferença entre termos consecutivos é constante. Se a_1 é o primeiro termo e r é a razão, o termo geral é:

$$a_n = a_1 + (n - 1)r, \quad n \in \mathbb{N}^*.$$

Exemplo: números pares

Sequência $(2, 4, 6, 8, \dots)$ com regra $a_n = 2n$.

```

1  from manim import *
2
3  class Pares(Scene):
4      def construct(self):
5
6          self.add(Text("Sequência dos pares",
7                        font_size=28).to_edge(UP))
8
9          grupo = VGroup()
10         for i in range(1, 5):
11             termo = MathTex(str(2 * i))
12             indice = MathTex(f"n={i}").scale(0.7)
13             indice.next_to(termo, DOWN, buff=0.15)
14             grupo.add(VGroup(termo, indice))
15
16         grupo.arrange(RIGHT, buff=1.2)
17
18         self.play(Write(grupo), run_time=3)
19         self.wait(1)    # ponto de pausa: "qual é a regra?"
20
21         regra = MathTex("a_n = 2n", color=YELLOW).scale(1.5)
22         regra.to_edge(DOWN)
23         self.play(Write(regra))
24         self.wait(2)

```

Código 14.1: D32 – Sequência dos pares com regra geral

Exemplo: números ímpares

Mesma estrutura, regra $a_n = 2n - 1$.

```

1  from manim import *
2
3  class Impares(Scene):
4      def construct(self):
5
6          self.add(Text("Sequência dos ímpares",
7                        font_size=28).to_edge(UP))
8
9          grupo = VGroup()
10         for i in range(1, 5):
11             termo = MathTex(str(2 * i - 1))
12             indice = MathTex(f"n={i}").scale(0.7)
13             indice.next_to(termo, DOWN, buff=0.15)
14             grupo.add(VGroup(termo, indice))
15
16         grupo.arrange(RIGHT, buff=1.2)
17         self.play(Write(grupo), run_time=3)
18         self.wait(1)
19
20         regra = MathTex("a_n = 2n-1", color=YELLOW).scale(1.5)
21         regra.to_edge(DOWN)
22         self.play(Write(regra))
23         self.wait(2)

```

Código 14.2: D32 – Sequência dos ímpares com regra geral



Dica

14.1

Use **f-strings** (como `f"n={i}"`) para gerar textos dinâmicos a partir de variáveis Python. Isso permite alterar o número de termos exibidos mudando apenas o `range(1, 5)` para `range(1, 7)`, por exemplo, sem reescrever nenhum outro trecho do código.

14.4 Sequências figurativas

Nas sequências figurativas, o acréscimo de cada novo termo é visível espacialmente. Usamos cor contrastante para destacar exatamente o que foi adicionado – tornando o mecanismo de crescimento um objeto visual.

Números triangulares



Definição

14.2

[Números triangulares] O n -ésimo número triangular T_n é a quantidade de pontos necessários para formar um triângulo com n pontos em cada lado:

$$T_n = \frac{n(n+1)}{2}, \quad n \in \mathbb{N}^*.$$

```

1  from manim import *
2
3  class Triangulares(Scene):
4      def construct(self):
5
6          self.add(Text("Números triangulares",
7                        font_size=28).to_edge(UP))
8
9          y = 2
10         for i in range(1, 5):
11             fila = VGroup(*[Dot(radius=0.12, color=BLUE)
12                            for _ in range(i)])
13             fila.arrange(RIGHT, buff=0.25)
14             fila.move_to([0, y, 0], LEFT)
15             self.play(Create(fila), run_time=0.6)
16
17             total = MathTex(
18                 str(i * (i + 1) // 2)
19             ).scale(0.7).next_to(fila, RIGHT, buff=0.3)
20             self.add(total)
21             y -= 0.9
22
23         regra = MathTex(
24             r"T_n = \frac{n(n+1)}{2}", color=YELLOW
25         ).to_edge(DOWN)
26         self.play(Write(regra))
27         self.wait(2)

```

Código 14.3: D32 – Números triangulares com acréscimo progressivo

Números quadrados

Na versão abaixo, os termos anteriores permanecem visíveis para que o aluno veja o padrão de crescimento acumulado – não apenas cada termo isolado.

```

1  from manim import *
2
3  class Quadrados(Scene):
4      def construct(self):
5
6          self.add(Text("Números quadrados:  $n^2$ ",
7                        font_size=28).to_edge(UP))
8
9          posicoes = [LEFT * 4.5, LEFT * 1.5,
10                     RIGHT * 1.5, RIGHT * 4.5]
11
12         for n, pos in enumerate(posicoes, start=1):
13
14             base = VGroup(*[
15                 Dot(radius=0.09, color=BLUE, fill_opacity=0.6)
16                 for _ in range(n * n)
17             ])
18             base.arrange_in_grid(n, n, buff=0.22)
19             base.move_to(pos)
20
21             indice = MathTex(
22                 f" $n={n}, \\ n^2={n**2}$ "
23             ).scale(0.55).next_to(base, DOWN, buff=0.2)
24
25             self.play(
26                 FadeIn(base),
27                 Write(indice),
28                 run_time=0.8
29             )
30             self.wait(0.5)
31
32             regra = MathTex(" $a_n = n^2$ ", color=YELLOW).scale(1.4)
33             regra.to_edge(DOWN)
34             self.play(Write(regra))
35             self.wait(2)

```

Código 14.4: D32 – Números quadrados com todos os termos visíveis

14.5 Durante a exibição

Pausa 1 – após os quatro termos aparecerem, antes da regra

Pause quando os quatro termos estiverem na tela mas a expressão algébrica ainda não tiver aparecido. Pergunte: “Alguém consegue descrever em palavras a regra que gerou esses números?” Não aceite apenas “vai aumentando” – peça precisão: “quanto aumenta de um termo para o seguinte? Sempre a mesma quantidade?”

Esta é a pausa mais importante do capítulo. É aqui que a visualização deixa de ser passiva e se torna objeto de raciocínio.

Pausa 2 – ao aparecer a expressão algébrica

Pause logo que a expressão surgir. Pergunte: “Verifiquem: para $n = 1$, a expressão dá quanto? E para $n = 3$?” Peça que os alunos substituam os valores antes de confirmar visualmente.

14.6 Depois de ver

Atividade de consolidação

Peça que os alunos façam duas coisas em sequência, nesta ordem:

1. **Descrever em palavras** a regra de formação de uma das sequências vistas. Exemplo: “A sequência dos pares cresce sempre de 2 em 2, então o n -ésimo termo é 2 vezes n .”
2. **Escrever a expressão algébrica** a partir da descrição em palavras.

A verbalização precede a simbolização. Um aluno que escreve $a_n = 2n$ sem conseguir explicar o que isso significa ainda não generalizou – apenas copiou um símbolo. A descrição em palavras é o teste de compreensão.

14.7 Desafios para o professor

1. Crie uma animação para a sequência dos múltiplos de 3 numa reta numérica, com o ponto saltando de 3 em 3 e o índice se atualizando a cada salto.
2. Adapte a animação dos triangulares para usar barras empilhadas em vez de pontos: cada fila é uma barra horizontal que cresce de comprimento.
3. Crie uma animação para a sequência não linear $(2, 4, 8, 16, \dots)$ com expressão $a_n = 2^n$. Destaque que o acréscimo não é constante – ele também dobra a cada termo.



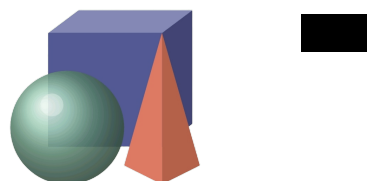
▶ **Video 14.1**

D32 – Sequências e padrões: do visual à expressão algébrica

https://youtu.be/INSEIRL_D32

15

Equações e inequações do 1º grau: D33



UMA EQUAÇÃO é uma afirmação de equilíbrio. Dois lados que pesam o mesmo. Quando o professor escreve $2x + 3 = 7$ no quadro e pede ao aluno que “resolva”, está pedindo que ele encontre o valor de x que mantém esse equilíbrio. Mas o aluno que aprende a resolver equações apenas como sequência de passos algébricos – “passa para o outro lado com sinal trocado” – opera sem imagem. Não sabe por que os passos funcionam. A metáfora da balança não é apenas uma ilustração: ela externaliza o princípio de igualdade como objeto visível. Quando você remove blocos dos dois lados simultaneamente, o aluno não apenas aplica uma regra – ele vê por que o equilíbrio se mantém.

15.1 Antes de exhibir

O que o aluno precisa saber

Esta animação pressupõe que o aluno opera com adição e subtração de números inteiros e que compreende o sinal de igualdade como equilíbrio, não apenas como “resultado de uma conta”. Um aluno que interpreta $=$ como “dá” tende a ver a equação como uma instrução de cálculo, não como uma afirmação que pode ser verdadeira ou falsa.

Antes de exhibir, verifique: se você disser “ $3 + 5 = 8$ ” e “ $3 + 5 = 9$ ”, o aluno consegue dizer qual é verdadeira e qual é falsa? Se consegue, a noção de igualdade está presente. Se

responder apenas “oito” para a primeira, sem avaliar a segunda, o conceito de igualdade como afirmação ainda precisa ser trabalhado antes da animação.

Atividade de aquecimento

Desenhe no quadro uma balança de pratos com 5 blocos de um lado e 5 do outro. Pergunte: “Está equilibrada?” Retire 2 de um lado apenas. “E agora?” Pergunte: “O que preciso fazer do outro lado para equilibrar de novo?”

Essa conversa prepara a animação sem revelar o procedimento algébrico – o aluno constrói a intuição do equilíbrio antes de ver a formalização.

15.2 Equações na balança

A equação $2x + 3 = 7$ pode ser vista como dois pratos em equilíbrio: no esquerdo, dois blocos vermelhos representando x e três blocos azuis representando 1; no direito, sete blocos azuis representando 7.



Definição

15.1

[Equação do 1º grau] Uma equação do primeiro grau na incógnita x é uma sentença matemática da forma $ax + b = c$, com $a \neq 0$, cuja solução é o valor de x que torna a igualdade verdadeira.

```

1  from manim import *
2
3  class EquacaoBalanca(Scene):
4      def construct(self):
5
6          self.add(Text("Equação:  $2x + 3 = 7$ ",
7                        font_size=28).to_edge(UP))
8
9          # Lado esquerdo: 2 blocos vermelhos (x) + 3 azuis (1)
10         esq = VGroup(
11             *[Square(side_length=0.6, color=RED,
12                     fill_opacity=0.7) for _ in range(2)],
13             *[Square(side_length=0.6, color=BLUE,
14                     fill_opacity=0.7) for _ in range(3)]
15         ).arrange(RIGHT, buff=0.15)
16         esq.move_to(LEFT * 3 + UP)
17
18         # Lado direito: 7 blocos azuis (constante 7)
19         dir_ = VGroup(*[
20             Square(side_length=0.6, color=BLUE,
21                   fill_opacity=0.7) for _ in range(7)
22         ])
23         dir_.arrange(RIGHT, buff=0.15)
24         dir_.move_to(RIGHT * 3 + UP)
25
26         # Balança: travessão e base triangular
27         travessao = Line(LEFT * 5, RIGHT * 5).shift(UP * 0.5)
28         base = Triangle().scale(0.5).next_to(travessao, DOWN)
29
30         self.play(Create(travessao), Create(base))
31         self.play(FadeIn(esq), FadeIn(dir_))
32         self.wait(1.5) # ponto de pausa 1 (ver abaixo)
33
34         # Subtrair 3 de cada lado: move blocos azuis para baixo
35         remover_esq = VGroup(*esq[2:5])
36         remover_dir = VGroup(*dir_[3:])
37         self.play(
38             remover_esq.animate.shift(DOWN * 2).set_opacity(0),
39             remover_dir.animate.shift(DOWN * 2).set_opacity(0),
40             run_time=1.5
41         )
42         self.wait(0.5) # ponto de pausa 2
43

```



Dica

15.1

A variável foi renomeada de `dir` para `dir_` porque `dir` é uma função interna do Python. Usar nomes reservados como variáveis não causa erro imediato, mas é má prática que pode gerar comportamentos inesperados. Sempre que precisar nomear “lado direito”, use `dir_` ou `lado_dir`.

O que você acabou de ver na animação tem um nome formal:



Teorema

15.1

[Princípio aditivo da igualdade] Se adicionarmos ou subtrairmos o mesmo número a ambos os membros de uma equação, a igualdade se mantém e o conjunto solução não se altera.

Na balança, “tirar o mesmo peso dos dois pratos” é exatamente esse princípio em ação. A animação mostra a operação como processo físico antes de nomeá-la como propriedade algébrica.

Durante a exibição – equações

Pausa 1 – com a balança montada, antes de remover blocos

Pause quando a balança e os blocos estiverem visíveis. Pergunte: “O que os blocos vermelhos representam? E os azuis? Quantos blocos existem em cada lado? A balança está equilibrada?”

Pausa 2 – após a remoção dos blocos, antes de mostrar $2x = 4$

Pause quando os três blocos azuis sumirem de cada lado. Pergunte: “O que ficou em cada prato agora? Isso ainda está equilibrado? Alguém consegue escrever a nova equação?”

Depois de ver – equações

Peça que os alunos resolvam $3x - 2 = 10$ no caderno, descrevendo cada passo em linguagem natural: “Adiciono 2 dos dois lados porque...” e “Divido os dois lados por 3 porque...”. A verbalização dos passos é a consolidação da compreensão – não o resultado.

15.3 Inequações na reta numérica

Inequações têm como solução intervalos, não valores únicos. A animação precisa mostrar dois elementos distintos: a extremidade (aberta ou fechada) e o intervalo que se estende a partir dela.



Definição

15.2

[Inequação do 1º grau] Uma inequação do primeiro grau usa os símbolos $>$, $<$, $\geq$ ou $\leq$ para comparar expressões lineares. O conjunto solução é o intervalo de valores que satisfaz a desigualdade.



Dica

15.2

A diferença entre bolinha aberta ($\circ$, valor não incluído) e bolinha fechada ($\bullet$, valor incluído) é um dos pontos de maior confusão dos alunos em inequações. Reserve um momento específico da exibição para perguntar: “Esse valor faz parte da solução ou não?” antes de revelar o tipo de bolinha.

Exemplo: $x > 2$

```
1 from manim import *
2
3 class InequacaoMaiorQue(Scene):
4     def construct(self):
5
6         self.add(Text("Inequação:  $x > 2$ ",
7                        font_size=28).to_edge(UP))
8
9         reta = NumberLine(
10             x_range=[-1, 6, 1],
11             length=10,
12             include_numbers=True
13         )
14         self.play(Create(reta))
15
16         # Bolinha aberta: valor 2 NÃO pertence à solução
17         bolinha = Circle(radius=0.12, color=BLUE,
18                          fill_opacity=0).set_stroke(BLUE, 3)
19         bolinha.move_to(reta.n2p(2))
20         self.play(Create(bolinha))
21         self.wait(1) # ponto de pausa: aberta ou fechada?
22
23         # Intervalo: linha grossa partindo de 2 para a direita
24         intervalo = Line(
25             reta.n2p(2.1), reta.n2p(5.5),
26             color=YELLOW, stroke_width=8
27         )
28         self.play(Create(intervalo))
29
30         solucao = MathTex(" $x > 2$ ").set_color(YELLOW).scale(1.2)
31         solucao.to_edge(DOWN)
32         self.play(Write(solucao))
33         self.wait(2)
```

Exemplo: $x \leq 3$

```
1 from manim import *
2
3 class InequacaoMenorOuIgual(Scene):
4     def construct(self):
5
6         self.add(Text("Inequação:  $x \leq 3$ ",
7                        font_size=28).to_edge(UP))
8
9         reta = NumberLine(
10             x_range=[-1, 6, 1],
11             length=10,
12             include_numbers=True
13         )
14         self.play(Create(reta))
15
16         # Dot preenchido: valor 3 PERTENCE à solução
17         bolinha = Dot(reta.n2p(3), color=BLUE, radius=0.12)
18         self.play(Create(bolinha))
19         self.wait(1) # ponto de pausa: aberta ou fechada?
20
21         # Intervalo: linha grossa indo para a esquerda
22         intervalo = Line(
23             reta.n2p(-0.5), reta.n2p(3),
24             color=YELLOW, stroke_width=8
25         )
26         self.play(Create(intervalo))
27
28         solucao = MathTex(r"x \leq 3").set_color(YELLOW).scale(1.2)
29         solucao.to_edge(DOWN)
30         self.play(Write(solucao))
31         self.wait(2)
```

Código 15.3: D33 – Inequação $x \leq 3$ na reta numérica

Durante a exibição – inequações

Pausa – após a bolinha aparecer, antes do intervalo

Esta é a pausa mais importante das animações de inequação. Pause quando a bolinha estiver visível e pergunte: “Esse ponto faz parte da solução ou não? Como a bolinha nos diz isso?” Depois de ouvir as respostas, exiba o intervalo.

Depois de ver – inequações

Proponha quatro situações para os alunos classificarem antes de representar: $x > -1$, $x \leq 4$, $x < 0$, $x \geq 2$. Para cada uma, peça que decidam: bolinha aberta ou fechada? Intervalo para a direita ou para a esquerda? Só então representem na reta. A decisão antes do desenho consolida a compreensão dos símbolos.

15.4 Desafios para o professor

1. Crie uma animação para a equação $3x - 2 = 10$ usando a mesma metáfora da balança. Quantos blocos de cada cor em cada prato?
2. Crie uma cena que mostre a inequação $-1 \leq x < 4$ na reta numérica, com bolinha fechada à esquerda e aberta à direita.
3. Crie uma animação que mostre o efeito de multiplicar uma inequação por -1 : o intervalo “vira” de lado e o símbolo muda. Por exemplo, $-x > 3$ equivale a $x < -3$. Mostre as duas retas em sequência e destaque a inversão do sentido.



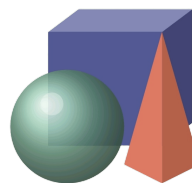
▶ Video 15.1

D33 – Equações na balança e inequações na reta

<https://youtu.be/INSEIRURLD33>

16

Sistemas de equações e representação geométrica: D34 e D35



UM SISTEMA de duas equações é duas condições que precisam ser satisfeitas ao mesmo tempo. Algebricamente, isso parece abstrato. Geometricamente, é imediato: duas retas no plano, e a solução é onde elas se encontram. A maior dificuldade do D35 não é algébrica – é a passagem entre dois modos de representar o mesmo objeto. A equação e a reta são a mesma coisa escrita de formas diferentes. Manter o sistema escrito visível enquanto as retas são traçadas ajuda o aluno a fazer essa conexão, que não acontece automaticamente quando os dois registros aparecem separados em momentos distintos.

16.1 Antes de exhibir

O que o aluno precisa saber

Este capítulo usa o plano cartesiano intensivamente. O aluno precisa localizar pontos, ler coordenadas com fluência (D9) e ter noção de que uma equação linear descreve uma reta. Se a turma não domina o plano cartesiano, vale uma revisão do D9 antes de avançar.

Atividade de aquecimento

Escreva no quadro a equação $x + y = 10$. Pergunte: “Quais pares de números somam 10?” Anote as respostas: $(1, 9)$, $(2, 8)$, $(3, 7)$, $(4, 6)$...

Depois pergunte: “Se eu plotar todos esses pontos no plano cartesiano, o que vai aparecer?” Deixe as respostas sem confirmar. A animação vai revelar que os pontos se alinham numa reta – e isso é o que a primeira equação representa geometricamente.

16.2 Do problema para as retas

Considere o problema: “A soma de dois números é 10 e a diferença entre eles é 2. Quais são esses números?” A representação algébrica é o sistema:

$$\begin{cases} x + y = 10 \\ x - y = 2 \end{cases}$$

Reescrevendo na forma resolvida: $y = 10 - x$ e $y = x - 2$.



Definição

16.1

[Sistema de equações do 1º grau] Um sistema de duas equações lineares com duas incógnitas tem a forma $a_1x + b_1y = c_1$ e $a_2x + b_2y = c_2$. A solução é o par ordenado (x, y) que satisfaz simultaneamente as duas equações.

Cada equação do sistema representa uma reta no plano cartesiano. Encontrar a solução do sistema significa encontrar o ponto onde as duas retas se cruzam.

```

1  from manim import *
2
3  class SistemaLinear(Scene):
4      def construct(self):
5
6          self.add(Text("Sistema de equações",
7                        font_size=26).to_edge(UP))
8
9          # Sistema escrito: permanece visível enquanto as retas surgem
10         sistema = MathTex(
11             r"\begin{cases} x + y = 10 \\",
12             r"x - y = 2 \end{cases}"
13         ).scale(0.9)
14         self.play(Write(sistema))
15         self.wait(1) # pausa: ler o sistema antes do gráfico
16
17         # Plano cartesiano
18         eixos = Axes(
19             x_range=[-1, 11, 1],
20             y_range=[-1, 11, 1],
21             x_length=7, y_length=7,
22             axis_config={"include_numbers": True}
23         ).to_edge(RIGHT, buff=0.3)
24         self.play(Create(eixos))
25         self.play(Write(MathTex("x").next_to(
26             eixos.x_axis.get_end(), RIGHT)),
27             Write(MathTex("y").next_to(
28                 eixos.y_axis.get_end(), UP)))
29
30         # Primeira reta: y = 10 - x (azul, como primeira equação)
31         reta1 = eixos.get_graph(lambda x: 10 - x, color=BLUE)
32         rot1 = MathTex("y = 10 - x", color=BLUE, font_size=22)
33         rot1.next_to(reta1.get_start(), UP, buff=0.2)
34         self.play(Create(reta1), Write(rot1))
35         self.wait(0.5) # pausa 1: apontar para a equação e a reta
36
37         # Segunda reta: y = x - 2 (verde, como segunda equação)
38         reta2 = eixos.get_graph(lambda x: x - 2, color=GREEN)
39         rot2 = MathTex("y = x - 2", color=GREEN, font_size=22)
40         rot2.next_to(reta2.get_start(), DOWN, buff=0.2)
41         self.play(Create(reta2), Write(rot2))
42         self.wait(1) # pausa 2: "onde as retas vão se cruzar?"
43

```



O ponto de interseção é calculado programaticamente a partir das duas equações, não fixado manualmente. Isso garante que, se você alterar as equações do sistema, o ponto se moverá automaticamente para a posição correta. Sempre que possível, calcule posições a partir das equações – não use coordenadas fixadas “de cabeça”.

Durante a exibição – sistema com solução única

Pausa 1 – após a primeira reta aparecer

Aponte fisicamente para a primeira equação escrita e para a reta azul. Pergunte: “Esta reta representa qual equação? Por que os pontos desta reta satisfazem $x + y = 10$?” Escolha um ponto visível da reta, por exemplo $(3, 7)$, e verifique com a turma: $3 + 7 = 10$? Sim.

Pausa 2 – com as duas retas visíveis, antes do ponto

Pause antes de destacar a interseção. Pergunte: “Onde vocês acham que as retas vão se cruzar? Marquem no caderno a estimativa.” Revelar o ponto após a previsão é muito mais eficaz do que revelar sem preparação.

Pausa 3 – após o ponto $(6, 4)$ aparecer

Pergunte: “Alguém verifica? O ponto $(6, 4)$ satisfaz a primeira equação? E a segunda?” Peça que substituam na reta: $6 + 4 = 10$? $6 - 4 = 2$? Essa verificação algébrica conecta o que foi visto geometricamente com o procedimento que o D34 exige.

Depois de ver – sistema com solução única

Proponha um novo sistema: $2x + y = 8$ e $x - y = 1$. Peça que os alunos sigam o mesmo processo: reescrevam na forma $y = \dots$, esbocem as retas no papel quadriculado, estimem a interseção e verifiquem algebricamente. Só depois exibam a animação criada com o código adaptado.

16.3 Os três casos: uma solução, infinitas, nenhuma



Teorema

16.1

[Classificação dos sistemas lineares] Um sistema de duas equações do primeiro grau pode ter:

- **Uma solução:** retas concorrentes (se cruzam num ponto).
- **Infinitas soluções:** retas coincidentes (são a mesma reta).
- **Nenhuma solução:** retas paralelas distintas (nunca se cruzam).

A representação geométrica torna essa classificação imediata: o aluno *vê* por que um sistema impossível não tem solução – as retas nunca se cruzam. E vê por que um sistema indeterminado tem infinitas – as retas são a mesma.

Retas coincidentes – infinitas soluções

```
1 from manim import *
2
3 class SistemaCoincidente(Scene):
4     def construct(self):
5
6         self.add(Text("Infinitas soluções",
7                        font_size=26).to_edge(UP))
8
9         eixos = Axes(
10             x_range=[-1, 6, 1], y_range=[-1, 6, 1],
11             x_length=7, y_length=7,
12             axis_config={"include_numbers": True}
13         ).center()
14         self.play(Create(eixos))
15
16         # Primeira reta:  $y = 5 - x$ 
17         reta1 = eixos.get_graph(lambda x: 5 - x, color=BLUE)
18         rot1 = MathTex("x + y = 5", color=BLUE, font_size=22)
19         rot1.next_to(reta1.get_start(), UP)
20         self.play(Create(reta1), Write(rot1))
21
22         # Segunda reta: mesma reta com transparência
23         # Transparência mostra que está sobreposta à primeira
24         reta2 = eixos.get_graph(lambda x: 5 - x,
25                                color=GREEN, stroke_opacity=0.5)
26         rot2 = MathTex("2x + 2y = 10", color=GREEN, font_size=22)
27         rot2.next_to(reta1.get_end(), DOWN)
28         self.play(Create(reta2), Write(rot2))
29         self.wait(2)
```

Código 16.2: D34/D35 – Retas coincidentes: infinitas soluções

Durante a exibição – retas coincidentes

Após exibir as duas retas, pergunte: “Onde é o ponto de interseção?” Espere a resposta. Os alunos dirão “não tem” ou “em todo lugar”. Conecte: “Quando as duas equações representam a mesma reta, qualquer ponto que satisfaz uma também satisfaz a outra. Por isso há infinitas soluções.”

Retas paralelas – nenhuma solução

```
1 from manim import *
2
3 class SistemaParalelo(Scene):
4     def construct(self):
5
6         self.add(Text("Nenhuma solução",
7                        font_size=26).to_edge(UP))
8
9         eixos = Axes(
10             x_range=[-1, 6, 1], y_range=[-1, 6, 1],
11             x_length=7, y_length=7,
12             axis_config={"include_numbers": True}
13         ).center()
14         self.play(Create(eixos))
15
16         # Primeira reta:  $y = 5 - x$ 
17         reta1 = eixos.get_graph(lambda x: 5 - x, color=BLUE)
18         rot1 = MathTex("x + y = 5", color=BLUE, font_size=22)
19         rot1.next_to(reta1.get_start(), UP)
20         self.play(Create(reta1), Write(rot1))
21
22         # Segunda reta:  $y = 7 - x$  (paralela - mesmo coeficiente angular)
23         reta2 = eixos.get_graph(lambda x: 7 - x, color=GREEN)
24         rot2 = MathTex("x + y = 7", color=GREEN, font_size=22)
25         rot2.next_to(reta2.get_start(), DOWN)
26         self.play(Create(reta2), Write(rot2))
27         self.wait(2)
```

Código 16.3: D34/D35 – Retas paralelas: nenhuma solução

Durante a exibição – retas paralelas

Após exibir as duas retas, pergunte: “Existe algum ponto que esteja sobre as duas retas ao mesmo tempo?” A resposta visual é imediata: não. Conecte com a álgebra: “Se tentarmos resolver algebricamente, chegamos a $5 = 7$, que é falso. A geometria nos mostra por quê – as retas nunca se cruzam.”

16.4 A ponte entre D34 e D35

O D34 pede que o aluno identifique um sistema que expressa um problema. O D35 pede que o aluno reconheça a relação entre o sistema algébrico e sua representação geométrica. A animação trabalha os dois ao mesmo tempo: o sistema permanece visível enquanto as retas são traçadas. Essa simultaneidade é o que cria a conexão entre os dois registros.

Durante a exibição do sistema com solução única, uma estratégia eficaz é apontar fisicamente para cada equação e para a reta correspondente enquanto ela surge: “Esta equação – esta reta. Aquela equação – aquela reta. Onde elas se encontram? Esse ponto satisfaz as duas equações ao mesmo tempo.” O gesto físico de apontar para os dois registros em sincronia materializa a conexão que o D35 exige.

16.5 Desafios para o professor

1. Crie uma animação para o sistema $2x + y = 8$ e $x - y = 1$. Calcule a solução programaticamente antes de exibir o ponto.
2. Crie uma cena com três planos cartesianos menores lado a lado, mostrando simultaneamente os três casos: solução única, infinitas soluções e nenhuma solução.
3. Crie uma animação que sincronize os passos algébricos da resolução por substituição com as mudanças visuais nas retas: cada passo algébrico é exibido ao lado do gráfico enquanto a reta correspondente é destacada.



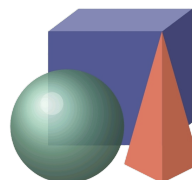
► Video 16.1

D34/D35 – Sistemas lineares: álgebra e geometria

https://youtu.be/INSERIR_URL_D34_D35

17

Outros tópicos do currículo



OS ONZE DESCRITORES trabalhados nos capítulos anteriores cobrem uma seleção representativa do currículo dos anos finais do Ensino Fundamental. Mas a metodologia que adotamos neste e-book – escolher conteúdos com natureza dinâmica ou transformacional, planejar com storyboard e incluir orientações de antes, durante e depois – é transferível para qualquer tema que envolva processos e variações.

Este capítulo apresenta quatro exemplos adicionais que demonstram essa transferibilidade. Cada conteúdo foi escolhido porque tem algo que a animação mostra melhor do que o quadro estático: a função afim tem coeficientes que alteram a inclinação e a posição da reta, e essa variação se entende melhor vendo a reta mudar do que comparando figuras separadas; o Teorema de Pitágoras tem uma relação de áreas que se torna evidente quando os quadrados são construídos sobre os lados do triângulo diante do aluno; os juros compostos têm crescimento exponencial, e a diferença entre crescimento linear e exponencial é visualmente imediata quando as duas curvas são traçadas na mesma tela; a probabilidade tem convergência de frequências, um conceito que só faz sentido depois de muitos lançamentos, e que a simulação pode mostrar em segundos.

17.1 Funções afim e quadrática

Conhecimento prévio necessário

O aluno precisa saber plotar pontos no plano cartesiano (D9) e ter noção de que uma equação linear corresponde a uma reta (D33–D35).

Gráfico estático da função afim

```
1 from manim import *
2
3 class FuncaoAfim(Scene):
4     def construct(self):
5
6         eixos = Axes(
7             x_range=[-3, 3, 1],
8             y_range=[-3, 5, 1],
9             x_length=7,
10            y_length=6,
11            axis_config={"include_numbers": True}
12        ).center()
13        self.play(Create(eixos))
14
15        # y = 2x + 1: coeficiente angular a=2, linear b=1
16        grafico = eixos.get_graph(
17            lambda x: 2 * x + 1, color=BLUE
18        )
19        rotulo = MathTex("y = 2x + 1", color=BLUE)
20        rotulo.next_to(grafico, UR, buff=0.2)
21        self.play(Create(grafico), Write(rotulo))
22        self.wait(2)
```

Código 17.1: Função afim: gráfico de $y = 2x + 1$



Varie os valores de a e b em animações separadas para mostrar como cada parâmetro afeta o gráfico. Isso constrói a intuição sobre o significado geométrico dos coeficientes sem precisar de definição prévia.

Variação do coeficiente angular com ValueTracker

Esta animação usa os recursos de D6 e D29: `ValueTracker` e `always_redraw`. A reta se redesenha a cada quadro enquanto o coeficiente angular muda.

```

1  from manim import *
2
3  class CoeficienteAngular(Scene):
4      def construct(self):
5
6          eixos = Axes(
7              x_range=[-3, 3, 1],
8              y_range=[-4, 4, 1],
9              x_length=7,
10             y_length=6,
11             axis_config={"include_numbers": True}
12         ).center()
13         self.add(eixos)
14
15         # ValueTracker: controla o coeficiente angular a
16         tracker = ValueTracker(1)
17
18         # Reta redesenhada a cada quadro
19         grafico = always_redraw(lambda:
20             eixos.get_graph(
21                 lambda x: tracker.get_value() * x,
22                 color=YELLOW
23             )
24         )
25
26         # Rótulo que mostra o valor atual de a
27         rotulo = always_redraw(lambda: MathTex(
28             f"y = {tracker.get_value():.1f}x",
29             color=YELLOW
30         ).to_corner(UR))
31
32         self.add(grafico, rotulo)
33         self.wait(1)
34
35         # a cresce de 1 para 3: reta fica mais inclinada
36         self.play(tracker.animate.set_value(3), run_time=3)
37         self.wait(1)
38
39         # a muda de sinal: reta passa por zero e inverte
40         self.play(tracker.animate.set_value(-2), run_time=4)
41         self.wait(2)

```

Atividade de consolidação – funções

Após as animações, proponha: “Sem usar o computador, esboce no papel o gráfico de $y = -x + 3$. Qual é a inclinação? Onde a reta cruza o eixo y ?” A estimativa manual antes da verificação com Manim consolida o significado geométrico dos coeficientes.

17.2 Teorema de Pitágoras

Conhecimento prévio necessário

O aluno precisa compreender área de quadrado ($A = l^2$) e ter noção de triângulo retângulo, cateto e hipotenusa.



Teorema

17.1

[Teorema de Pitágoras] Em todo triângulo retângulo, $a^2 + b^2 = c^2$, onde a e b são os catetos e c é a hipotenusa.

A animação constrói os quadrados com arestas coincidentes com os lados do triângulo, garantindo precisão geométrica.

```

1  from manim import *
2
3  class TeoremaPitagoras(Scene):
4      def construct(self):
5
6          titulo = Text("Teorema de Pitágoras",
7                          font_size=28).to_edge(UP)
8          self.play(Write(titulo))
9
10         # Triângulo retângulo com vértices exatos
11         # cateto horizontal = 4, vertical = 3, hipotenusa = 5
12         P1 = np.array([-2, -1, 0]) # canto inf. esquerdo
13         P2 = np.array([ 2, -1, 0]) # canto inf. direito
14         P3 = np.array([-2,  2, 0]) # canto sup. esquerdo
15
16         triangulo = Polygon(P1, P2, P3,
17                             color=WHITE, stroke_width=3)
18         self.play(Create(triangulo))
19         self.wait(0.5)
20
21         # Quadrado sobre cateto vertical (comprimento 3)
22         quad_vert = Polygon(
23             P1, P3,
24             P3 + np.array([-3, 0, 0]),
25             P1 + np.array([-3, 0, 0]),
26             color=BLUE, fill_opacity=0.3
27         )
28         rot_v = MathTex("a^2", color=BLUE).move_to(
29             np.array([-3.5, 0.5, 0]))
30         self.play(Create(quad_vert), Write(rot_v))
31         self.wait(0.4)
32
33         # Quadrado sobre cateto horizontal (comprimento 4)
34         quad_hor = Polygon(
35             P1, P2,
36             P2 + np.array([0, -4, 0]),
37             P1 + np.array([0, -4, 0]),
38             color=GREEN, fill_opacity=0.3
39         )
40         rot_h = MathTex("b^2", color=GREEN).move_to(
41             np.array([0, -3.5, 0]))
42         self.play(Create(quad_hor), Write(rot_h))
43         self.wait(0.4)

```


Durante a exibição – Pitágoras

Pausa – com os três quadrados visíveis, antes da fórmula

Pergunte: “Qual é a área do quadrado azul? E do verde? E do amarelo?” Para o triângulo 3-4-5: $9 + 16 = 25$. Peça que verifiquem numericamente antes de ver a fórmula aparecer.

Atividade de consolidação – Pitágoras

Proponha um triângulo com catetos 5 e 12. Peça que calculem a hipotenusa antes de exibir a animação com os novos valores. Depois adaptem o código substituindo as coordenadas dos vértices.

17.3 Juros compostos

Conhecimento prévio necessário

O aluno precisa compreender porcentagem como razão (D28) e ter noção de função – o montante depende do número de períodos.



Definição

17.1

[Juros compostos] $C(t) = C_0 \cdot (1 + i)^t$, onde C_0 é o capital inicial, i é a taxa por período e t é o número de períodos.

A animação compara juros simples e compostos na mesma tela, tornando visível a divergência entre crescimento linear e exponencial.

```

1  from manim import *
2
3  class JurosCompostos(Scene):
4      def construct(self):
5
6          eixos = Axes(
7              x_range=[0, 12, 2],
8              y_range=[0, 4, 1],
9              x_length=9,
10             y_length=5,
11             axis_config={"include_numbers": True}
12         ).center()
13         rot_x = Text("períodos", font_size=18).next_to(
14             eixos.x_axis.get_end(), RIGHT, buff=0.2)
15         rot_y = Text("montante", font_size=18).next_to(
16             eixos.y_axis.get_end(), UP, buff=0.2)
17         self.play(Create(eixos), Write(rot_x), Write(rot_y))
18
19         # Juros simples: crescimento linear  $C = 1 + 0.1t$ 
20         simples = eixos.get_graph(
21             lambda t: 1 + 0.1 * t, color=BLUE,
22             x_range=[0, 11]
23         )
24         rot_s = MathTex(
25             r"C_s = 1 + 0{,}1t", color=BLUE, font_size=22
26         ).next_to(simples.get_end(), LEFT, buff=0.2)
27
28         # Juros compostos: crescimento exponencial  $C = 1.1^t$ 
29         compostos = eixos.get_graph(
30             lambda t: 1.1 ** t, color=GREEN,
31             x_range=[0, 11]
32         )
33         rot_c = MathTex(
34             r"C_c = (1{,}1)^t", color=GREEN, font_size=22
35         ).next_to(compostos.get_end(), RIGHT, buff=0.1)
36
37         # Exibição sequencial para comparação
38         self.play(Create(simples), Write(rot_s))
39         self.wait(1)
40         self.play(Create(compostos), Write(rot_c))
41         self.wait(2)

```

Durante a exibição – juros

Pausa – após as duas curvas aparecerem

Pergunte: “Em qual período as curvas se cruzam? Antes disso, qual montante é maior? E depois?” Conecte com situações reais: por que juros compostos em dívida são perigosos a longo prazo.

Atividade de consolidação – juros

Com capital de R\$1.000 e taxa de 10% ao período, peça que os alunos calculem o montante para $t = 1, 2, 3, 5$ com juros simples e compostos. Depois verifiquem plotando os pontos sobre a animação.

17.4 Probabilidade

Conhecimento prévio necessário

O aluno precisa compreender probabilidade clássica e a noção de frequência relativa como razão entre resultados favoráveis e total de tentativas.



Definição

17.2

[Probabilidade clássica] $P(E) = \frac{\text{casos favoráveis}}{\text{casos possíveis}}$. Para um dado justo de 6 faces, $P(\text{cada face}) = 1/6$.

A animação simula lançamentos reais e atualiza o gráfico de barras a cada rodada, mostrando a convergência das frequências para os valores teóricos.

```

1  from manim import *
2  import random
3
4  class SimulacaoDados(Scene):
5      def construct(self):
6
7          titulo = Text("Simulação: 30 lançamentos",
8                        font_size=24).to_edge(UP)
9          self.add(titulo)
10
11         # Eixos para o gráfico de barras
12         eixos = Axes(
13             x_range=[0.5, 7, 1],
14             y_range=[0, 12, 2],
15             x_length=8,
16             y_length=4,
17             axis_config={"include_numbers": False}
18         ).shift(DOWN * 0.5)
19         self.add(eixos)
20
21         # Rótulos das faces
22         for i in range(1, 7):
23             lbl = Text(str(i), font_size=18).next_to(
24                 eixos.coords_to_point(i, 0), DOWN, buff=0.2)
25             self.add(lbl)
26
27         # Linha de referência teórica: 30/6 = 5
28         linha_teo = DashedLine(
29             eixos.coords_to_point(0.5, 5),
30             eixos.coords_to_point(6.5, 5),
31             color=YELLOW, dash_length=0.2
32         )
33         lbl_teo = MathTex(r"1/6", color=YELLOW,
34                          font_size=18).next_to(
35             eixos.coords_to_point(6.5, 5), RIGHT, buff=0.1)
36         self.add(linha_teo, lbl_teo)
37
38         # Simulação
39         contagens = [0] * 7 # índices 1 a 6
40         barras_grupo = VGroup()
41         self.add(barras_grupo)
42
43         for lancamento in range(30):

```



A linha tracejada amarela representa o valor teórico (5 lançamentos por face em 30 tentativas). Observar que as barras se aproximam dessa linha ao final é exatamente a lei dos grandes números tornada visível. Com 100 lançamentos, a convergência é ainda mais evidente – altere apenas o `range(30)` para `range(100)`.

Durante a exibição – probabilidade

Pausa – após a simulação completa

Pergunte: “Alguma face ficou exatamente em 5? Por quê?” e “Se repetirmos com 300 lançamentos, o que esperamos que aconteça?” Conecte com o conceito de frequência relativa como estimativa da probabilidade teórica.

Atividade de consolidação – probabilidade

Execute a simulação três vezes com a turma observando. Registre os resultados das três execuções. Peça que os alunos calculem a frequência relativa de cada face nas três execuções combinadas e comparem com $1/6$.

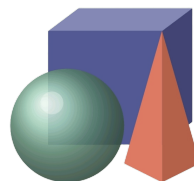


▶ Video 17.1

Cap. 15 – Outros tópicos: funções, Pitágoras, juros e probabilidade

<https://youtu.be/INSERIRURLCAP15>

Sobre os Autores





Emily Mayre Souza Moura da Silva

Professora da Educação Básica

Escola Estadual

Secretaria de Educação de MT

(SEDUC-MT)

Campus de Barra do Bugres, MT, Brasil

Formação:

- Mestrado em Matemática para a Educação Básica, *UNEMAT*, 2026
- Licenciatura em Matemática, *UNEMAT*, 2019

Interesses de pesquisa: Visualização matemática com auxílio da biblioteca

Manim.

`emilly@email.com`



Junior Cesar Alves Soares

Professor Adjunto IV na UNEMAT

Faculdade de Ciências Exatas e

Tecnológicas

UNEMAT

Barra do Bugres, MT, Brasil

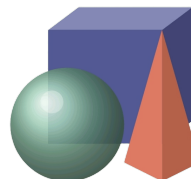
Formação:

- Doutorado em Matemática Aplicada, *UNICAMP*, 2016
- Mestrado em Matemática Aplicada, *UNICAMP*, 2011
- Licenciatura em Matemática, *UFMT*, 2009

Interesses de pesquisa: Cálculo Fracionário, Simetrias de Lie, Equações Diferenciais Fracionárias, Leis de conservação.

`juniorcasoares@unemat.br`

Referências Bibliográficas



- [1] BORBA, M. C., AND VILLARREAL, M. E. *Humans-with-media and the reorganization of mathematical thinking*. Springer, New York, 2005.
- [2] BULUBAAN, S., JUANDI, D., PUTRI, N., AND RACMI, C. N. The utilization of videos as learning media in mathematics learning: A systematic literature review. *Research and Development Journal of Education* 11, 2 (2025), 1093–1105.
- [3] DUVAL, R. *Registros de representação semiótica e aprendizagem matemática*. Sociedade Brasileira de Educação Matemática, São Paulo, 2009.
- [4] MACNAB, J. S., PHILLIPS, L. M., AND NORRIS, S. P. Visualizations and visualization in mathematics education. In *Reading for Evidence and Interpreting Visualizations in Mathematics and Science Education*, S. P. Norris, Ed. Sense Publishers, Rotterdam, 2012, pp. 103–122.
- [5] MANIM COMMUNITY. Manim: Mathematical animation engine, 2026. Acessado em: 1 abr. 2026.
- [6] PHILLIPS, L. M., NORRIS, S. P., AND MACNAB, J. S. *Visualization in Mathematics, Reading and Science Education*, vol. 5 of *Models and Modeling in Science Education*. Springer, Dordrecht, 2010.
- [7] PRESMEG, N. C. Visualization in high school mathematics. *For the Learning of Mathematics* 6, 3 (1986), 42–46.
- [8] TEN VOORDE, A., PIROI, M., AND BOS, R. A taxonomy of didactic roles of dynamic visualization in animated mathematics videos. *Teaching Mathematics and its Applications: An International Journal of the IMA* 44, 1 (2025), 47–67.
- [9] WING, J. M. Computational thinking. *Communications of the ACM* 49, 3 (2006), 33–35.